

7. 图像分割 (Image Segmentation)

邓 欣

Email: cindydeng@buaa.edu.cn

办公室：新主楼F921

本章内容

- ◆ 7.1 概 述
- ◆ 7.2 边缘检测算子
- ◆ 7.3 图像阈值分割
- ◆ 7.4 区域增长法和分开-合并区域方法
- ◆ 7.5 数学形态学在图像分割中的应用
- ◆ 7.6 基于UNET的智能分割处理

7.5.1 数学形态学历史

- ◆ **数学形态学**(mathematical morphology, MM): 是以集合论、格论为数学基础, 受生物学形态学思想启发发展而来的非线性图像处理与分析理论, 在图像处理、模式识别、计算机视觉等领域有广泛且成功的应用。
- ◆ 用小模板来探测图像, 提取图像区域的形状特征, 如边界、骨架和凸壳等
- ◆ **MM方法的优势:**
 - 以结构元素为工具, 形态学算子可在有效滤除噪声的同时, 完整保留目标的关键几何结构信息;
 - MM算法基于局部邻域运算, 天然适合并行处理, 且硬件实现容易;
 - 基于MM的边缘信息提取, 相比传统微分算子, 抗噪性更强, 且提取的边缘更连续、光滑;
 - 基于MM方法提取的图像骨架, 连续性好、断点少, 能完整保留目标的拓扑结构。

7.5.2 数学形态学的数学基础

◆ 集合论的一些基本概念：

— 属于、不属于、空集

- 令 A 是 Z^2 中的一个集合，如果 a 是其中的一个元素，称 a 属于 A ，并记作： $a \in A$ ，否则，称 a 不属于 A ，记为： $a \notin A$ ，如 A 中没有任何元素，称 A 为空集： $\emptyset$

— 子集、并集、交集

- $A \subseteq B, C = A \cup B, C = A \cap B$

— 不相连（互斥）、补集、差集

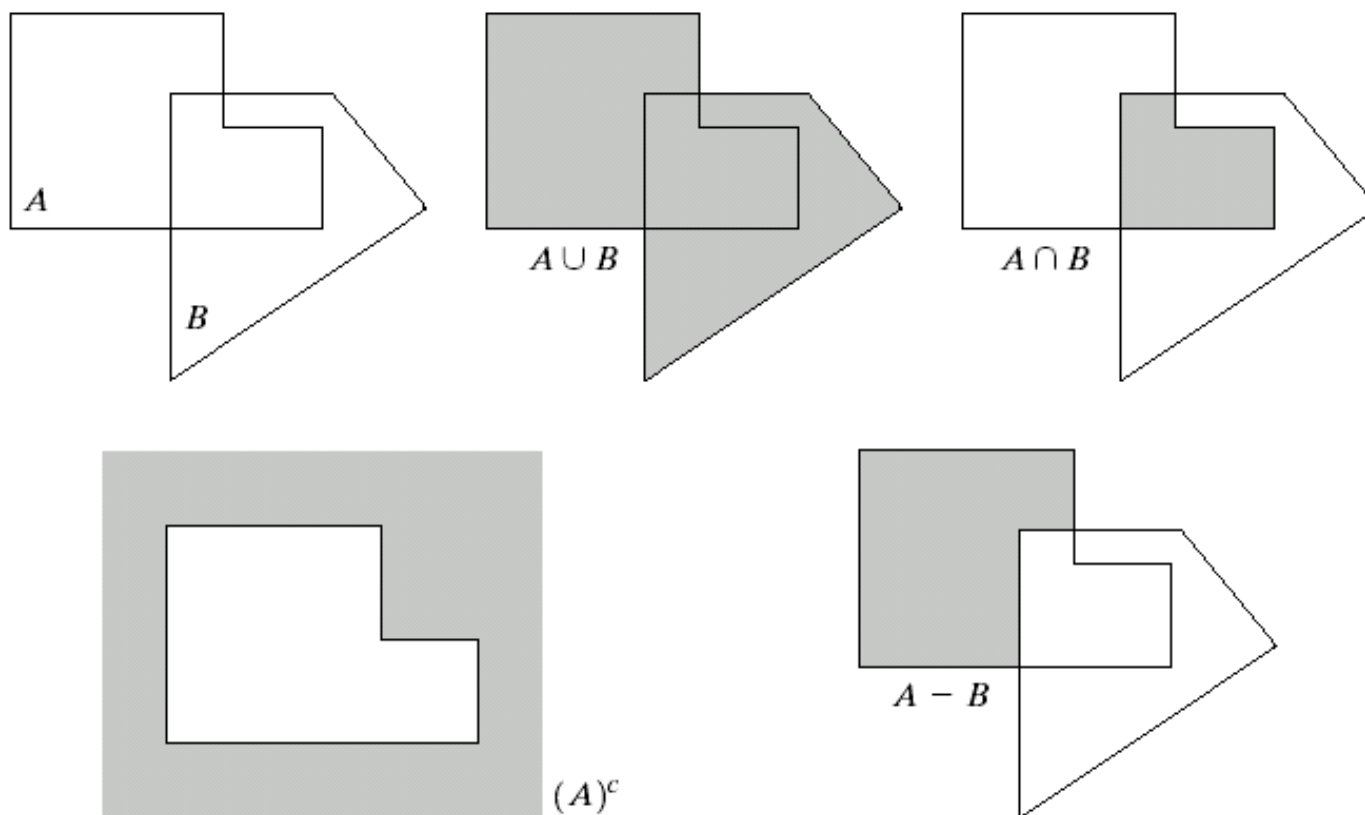
- $A \cap B = \emptyset, (A)^c = \{ a \mid a \notin A \},$
- $A - B = \{ c \mid c \in A, c \notin B \} = A \cap (B)^c$

— 反射 $\hat{A}_B = \{ w \mid w = -b, b \in B \}$ 即关于原集合原点对称

— 移位 $(A)_z = \{ c \mid c = a + z, a \in A \}$

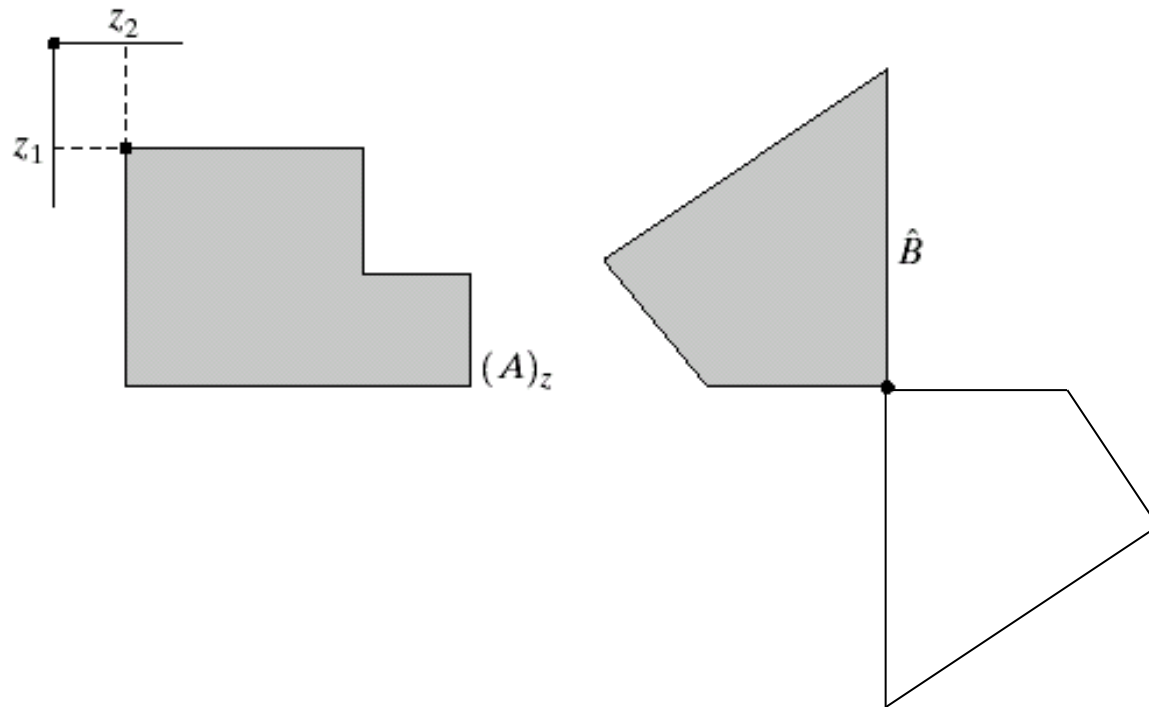
7.5.2 数学形态学的数学基础

- 并、交、补、减：



7.5.2 数学形态学的数学基础

- 移位、反射：



7.5.2 数学形态学的数学基础

◆ 二值图像中的基本逻辑操作：

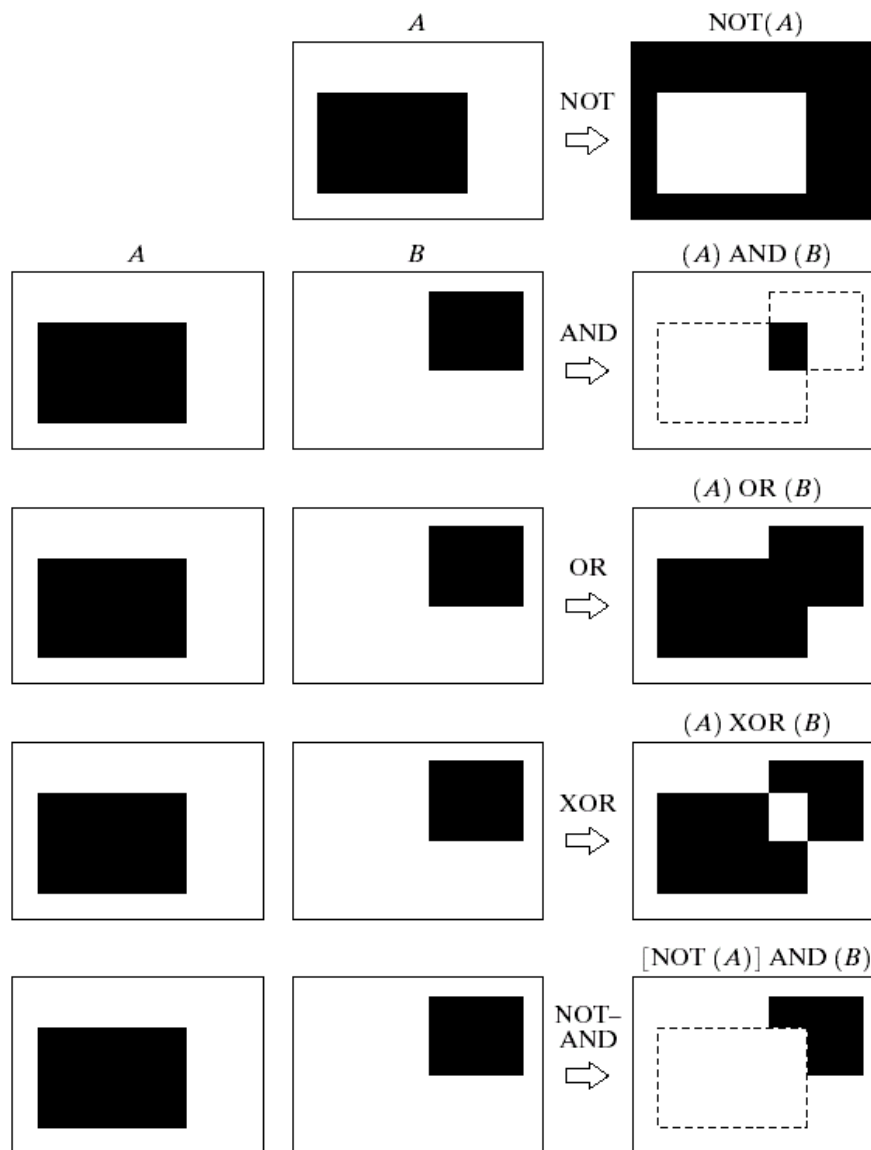
三种最基本的逻辑运算（功能完整的）：与、或、非（补）

p	q	$p \text{ AND } q \text{ (also } p \cdot q \text{)}$	$p \text{ OR } q \text{ (also } p + q \text{)}$	$\text{NOT } (p) \text{ (also } \bar{p} \text{)}$
0	0	0	0	1
0	1	0	1	1
1	0	0	1	0
1	1	1	1	0

尽管逻辑操作与集合操作间存在一一对应的关系，但逻辑操作只是针对二值图像。

7.5.2 数学形态学的数学基础

- ◆ 基本逻辑操作的图形表示：



7.5.3 二值数学形态学的基本运算

- ◆ 膨胀 (dilation)
- ◆ 腐蚀 (erosion)
- ◆ 开和闭 (opening and closing)



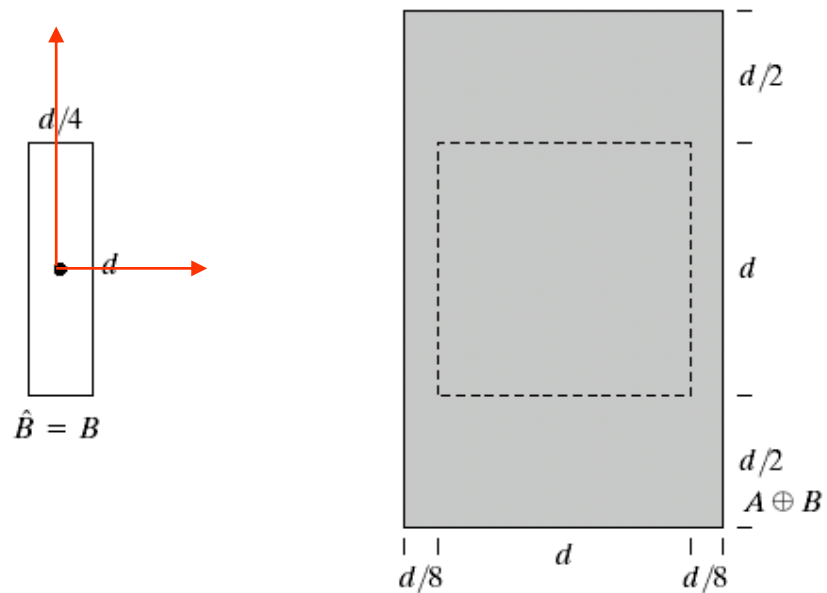
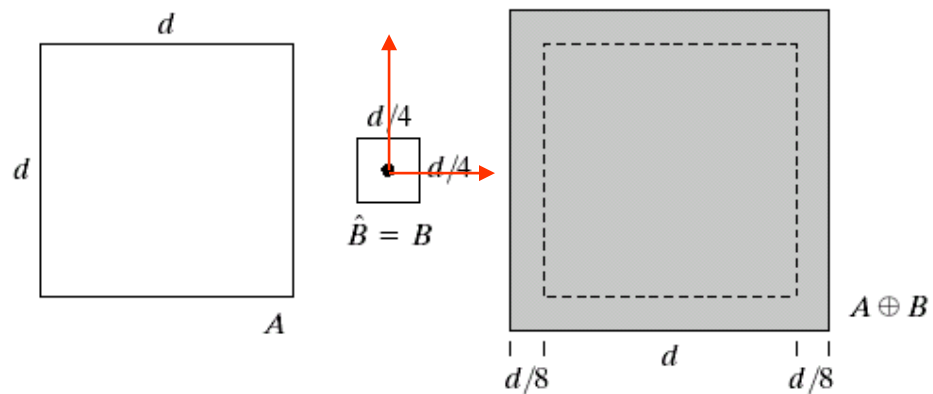
7.5.3.1 膨胀运算

- ◆ 假定A和B是 Z^2 上的两个集合，把A被B（结构元素）膨胀定义为：

$$A \oplus B = \left\{ z \mid \left(\mathcal{B} \right)_z \cap A \neq \emptyset \right\}$$

- ◆ **含义：**膨胀结果是把邻接背景与目标的**背景像素置**为目标像素，增加目标面积的作用。

7.5.3.1 膨胀运算



膨胀过程解释图示

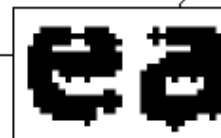
7.5.3.1 膨胀运算

◆ 膨胀运算应用举例

Historically, certain computer programs were written using only two digits rather than four to define the applicable year. Accordingly, the company's software may recognize a date using "00" as 1900 rather than the year 2000.



Historically, certain computer programs were written using only two digits rather than four to define the applicable year. Accordingly, the company's software may recognize a date using "00" as 1900 rather than the year 2000.



0	1	0
1	1	1
0	1	0

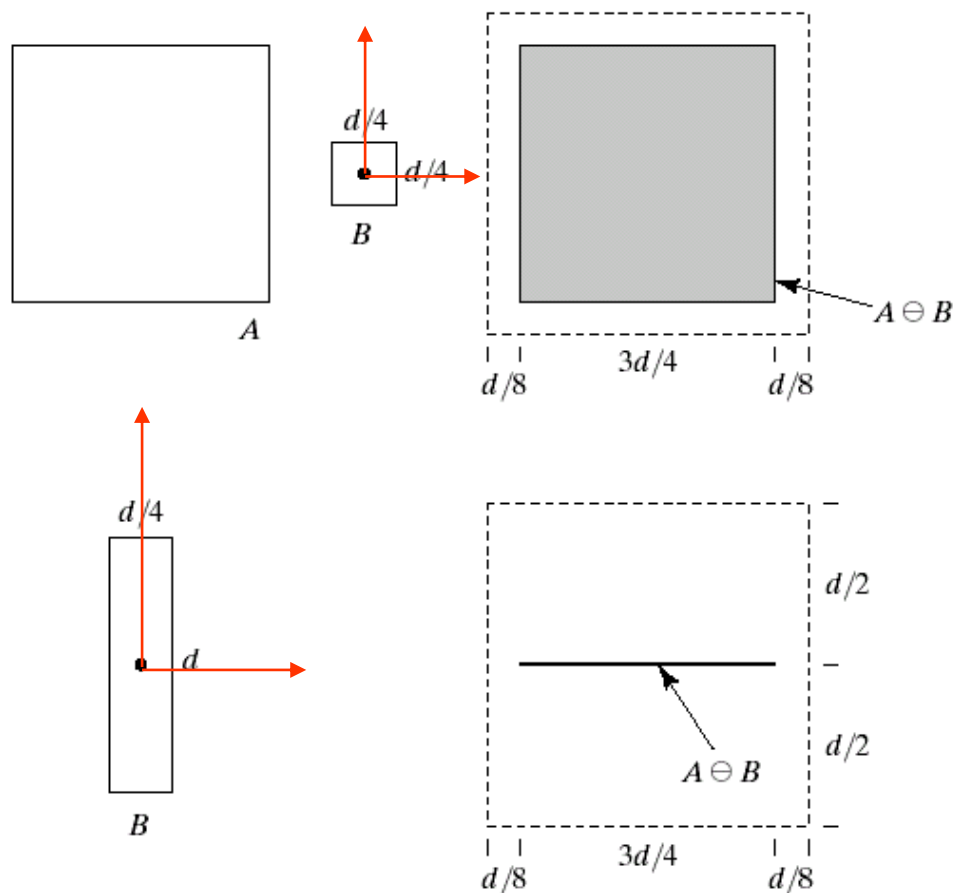
7.5.3.2 腐蚀运算

- 假定A和B是 Z^2 上的两个集合，把A被B腐蚀定义为：

$$A \$ B = \left\{ z \mid (B)_z \subseteq A \right\}$$

含义：腐蚀结果是把临近背景的**目标像素值**置为背景来达到缩小目标面积的作用。腐蚀为膨胀结果的补集

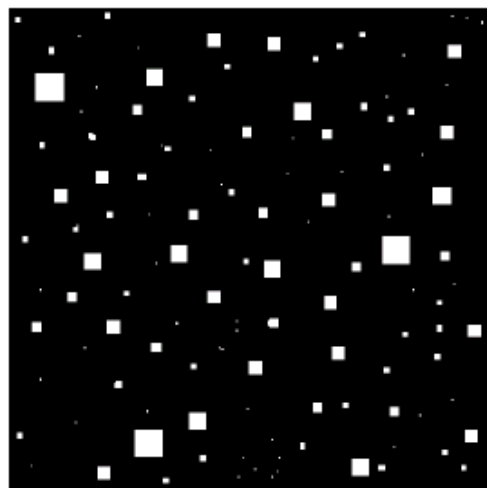
7.5.3.2 腐蚀运算



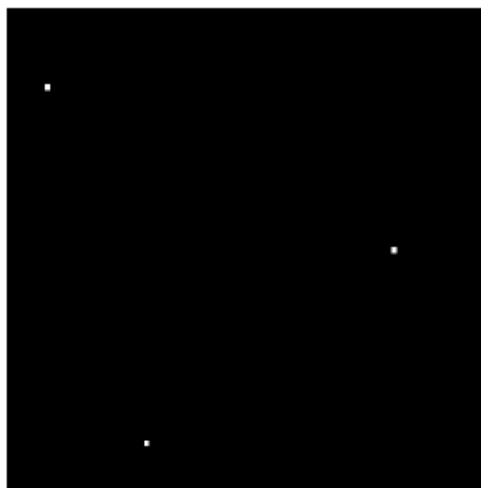
腐蚀过程解释图示

7.5.3.2 腐蚀运算

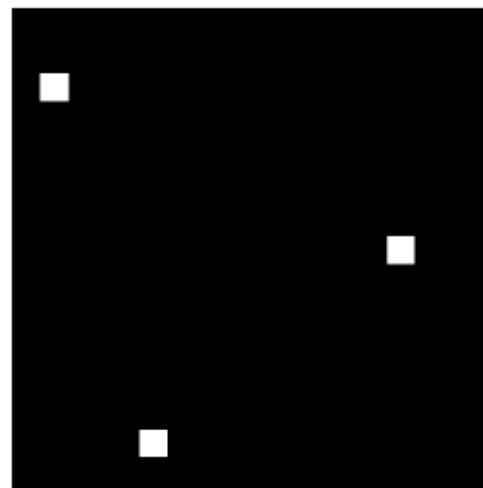
- ◆ 膨胀、腐蚀运算应用举例



边长为1,3,5,7,9和
15像素正方形



使用 13×13 像素大小
的结构元素腐蚀



使用 13×13 像素大小的
结构元素膨胀

本例中“**细节**”是从尺寸大小的角度讲的，同时该例中出现的一些概念是形态学滤波的基础。

7.5.3.2 腐蚀运算

- ◆ 腐蚀与膨胀的对偶性：

$$(A \$ B)^c = A^c \oplus B$$

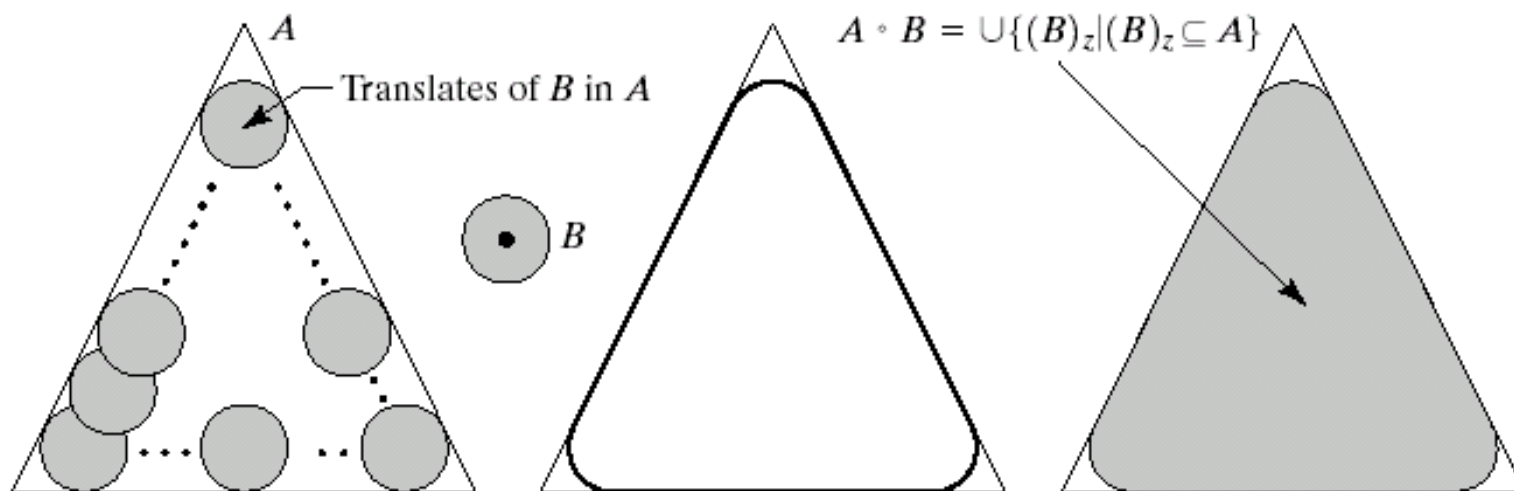
$$(A \oplus B)^c = A^c \$ B$$

图像A 被结构元素 B 腐蚀的补集，等于图像 A 的补集被结构元素B 的反射膨胀；反之，图像A 被 结构元素B 膨胀的补集，等于图像 A 的补集被结构元素 B 的反射腐蚀。

7.5.3.3 开运算

- 先用结构元B对A腐蚀，再对腐蚀结果用同样的结构元进行膨胀操作：

$$A \circ B = (A \ominus B) \oplus B$$



可见，开运算的边界是由这样一些点组成的，就是当 B 沿 A 的内部边界滚动时， B 中所能达到的 A 的内部边界的最远的点。

7.5.3.3 开运算

- 先用结构元B对A腐蚀，再对腐蚀结果用同样的结构元进行膨胀操作：

$$A \circ B = (A \ \$ \ B) \oplus B$$

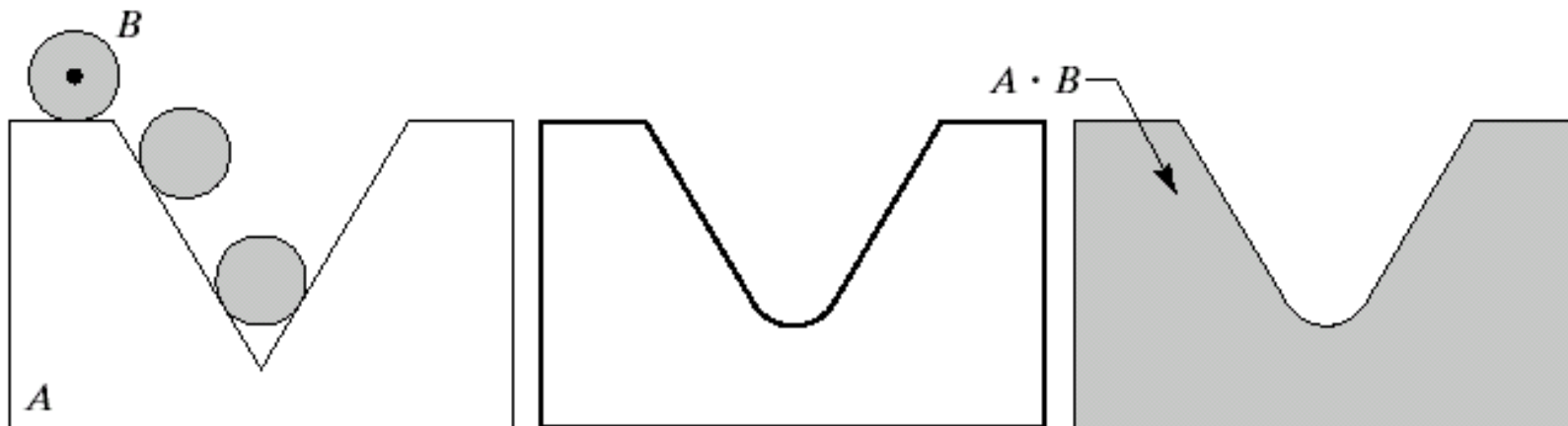
基本属性：

- ① 开的结果是A的子集；
- ② 如C是D的子集，则C与B开运算的结果是D与B开运算结果的子集；
- ③ 对同样的A，做多次开运算的结果与做一次是一样的；
- ④ 开运算用于去除细小突出、平滑轮廓。

7.5.3.4 闭运算

- 先用结构元B对A进行膨胀，再对膨胀结果用同样的结构元进行腐蚀操作，过程与开运算正好相反：

$$A \bullet B = (A \oplus B) \ominus B$$



可见，闭运算的边界是由这样一些点组成的，就是当结构元素B沿A的外部边界滚动时，B中所能达到的A的外部边界的最远点。

7.5.3.4 闭运算

- 先用结构元B对A进行膨胀，再对膨胀结果用同样的结构元进行腐蚀操作，过程与开运算正好相反：

$$A \bullet B = (A \oplus B) \otimes B$$

基本属性：

- ① A是闭运算结果的子集；
- ② 如C是D的子集，则C与B闭作用的结果是D与B闭运算结果的子集；
- ③ 对同样的A，做多次闭运算的结果与做一次是一样的；
- ④ 闭运算用于填充细小孔洞和凹槽。

7.5.3.4 闭运算

- ◆ 开运算与闭运算的对偶性：

$$(A \circ B)^c = A^c \bullet B$$

$$(A \bullet B)^c = A^c \circ B$$

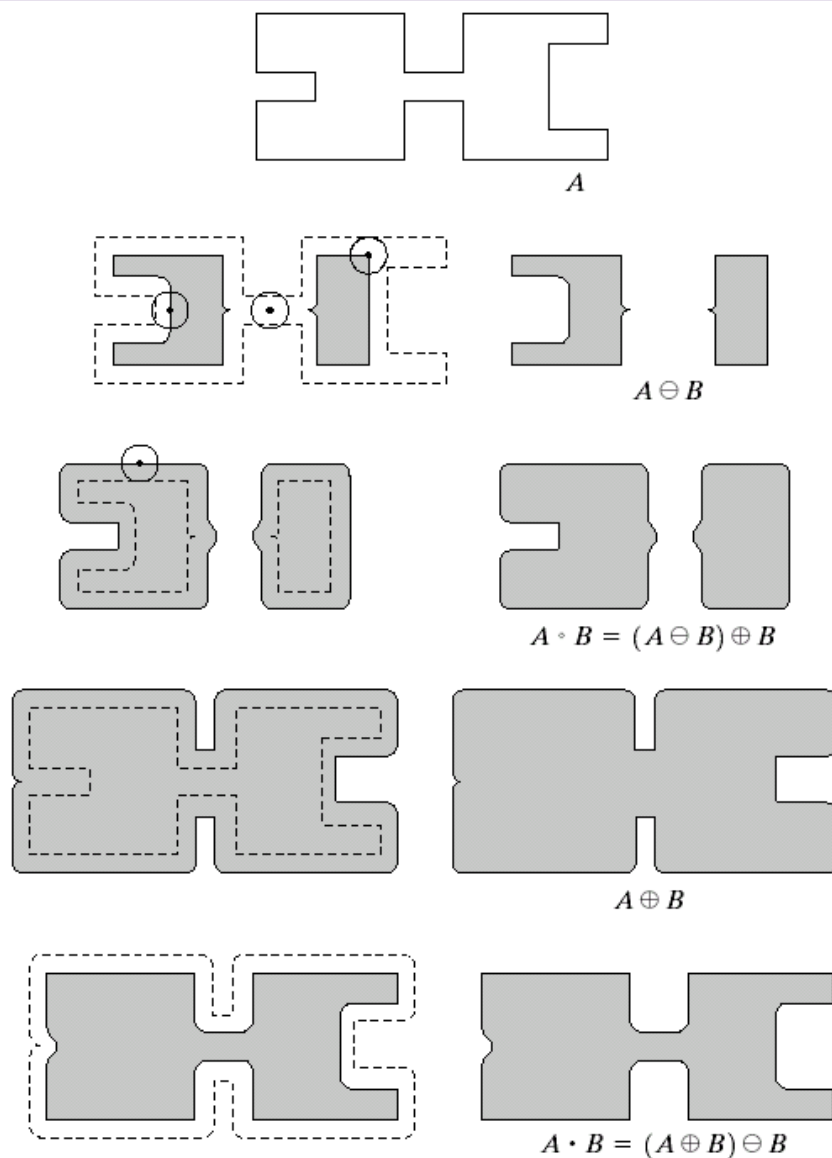
- 图像 A 被 B 做开运算的补集，等于 A 的补集被 B 的反射做闭运算；
- 图像 A 被 B 做闭运算的补集，等于 A 的补集被 B 的反射做开运算；
- 开运算与闭运算在补集下互为对偶。

7.5.3.4 闭运算

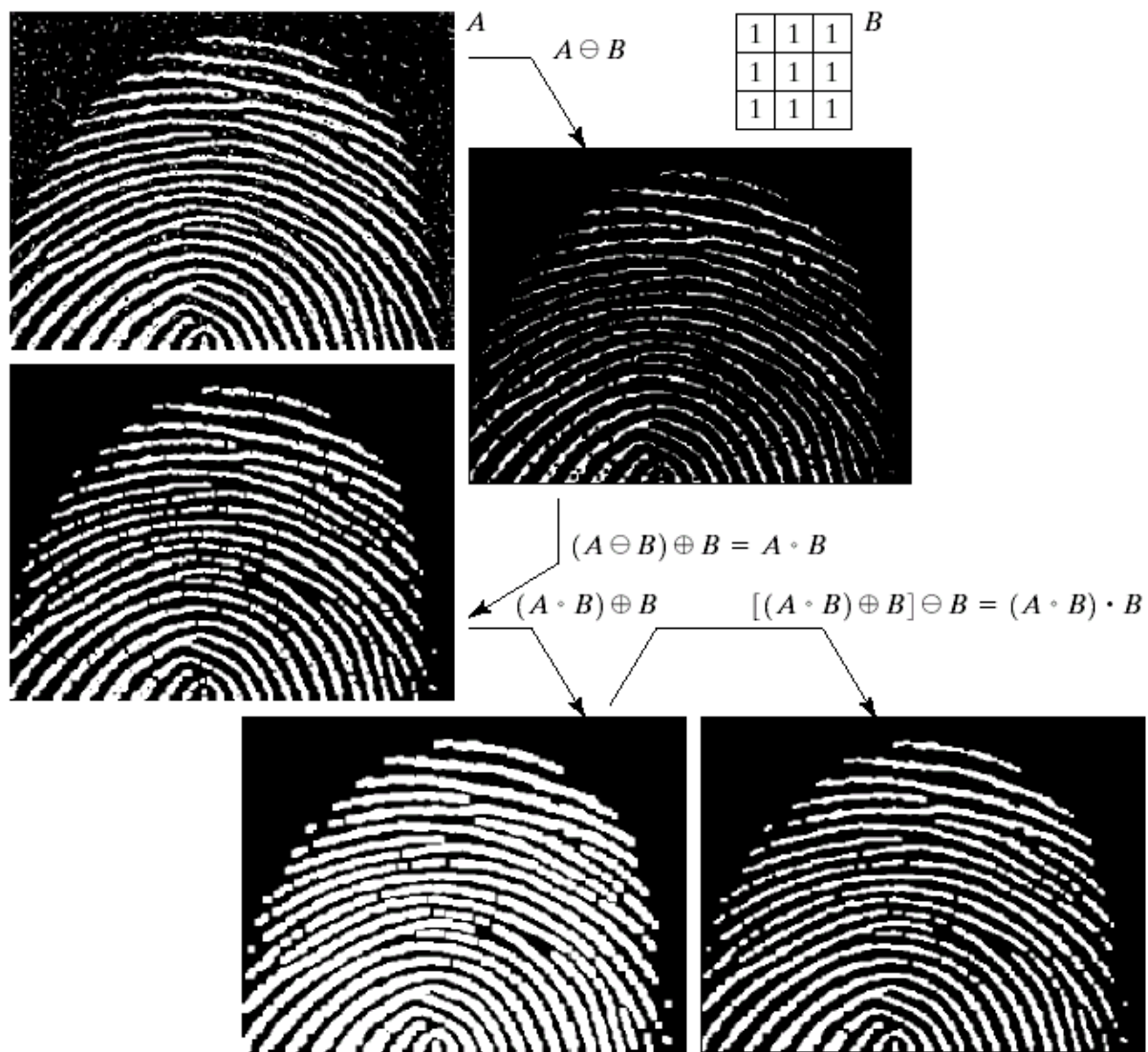
两种操作的大致效果如下：

➤ **开运算**：通常对图像轮廓进行平滑，使狭窄的“地峡”形状断开，去掉细的突起。

➤ **闭运算**：也是趋向于平滑图像的轮廓，但与开运算相反，它一般使窄的断开部位和细长的沟熔合，填补轮廓上的间隙。



7.5.3.4 闭运算



过程:

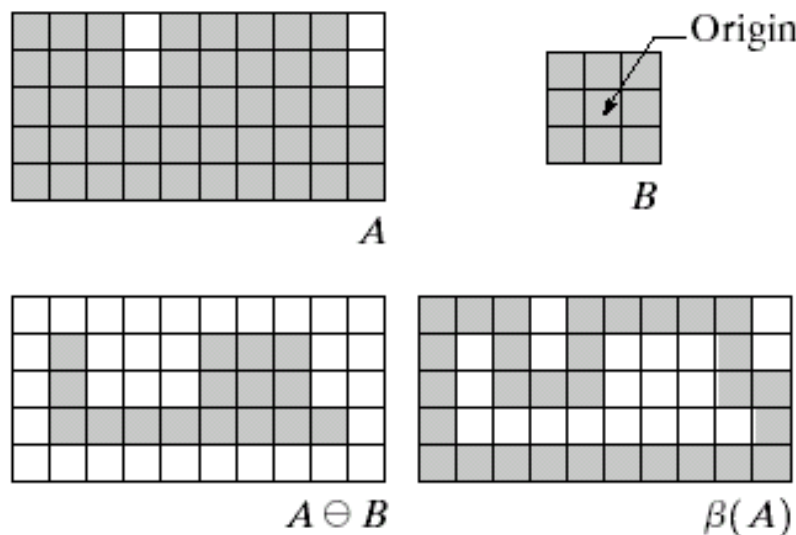
先开后闭，开消除噪声，闭修复开运算造成的指纹断裂。

7.5.4 二值形态学图像处理的基本操作

- ◆ **边界抽取：** 令集A的边界为 $\beta(A)$ ，其可以用某一合适的结构元素B对A先进行腐蚀，然后再把A减去腐蚀的结果来获得

$$\beta(A) = A - (A \ominus B)$$

- ◆ **图例说明**（所有的格子图像均用阴影表示1，白色表示0），当结构元素大小为 3×3 时，边界厚度为1像素。

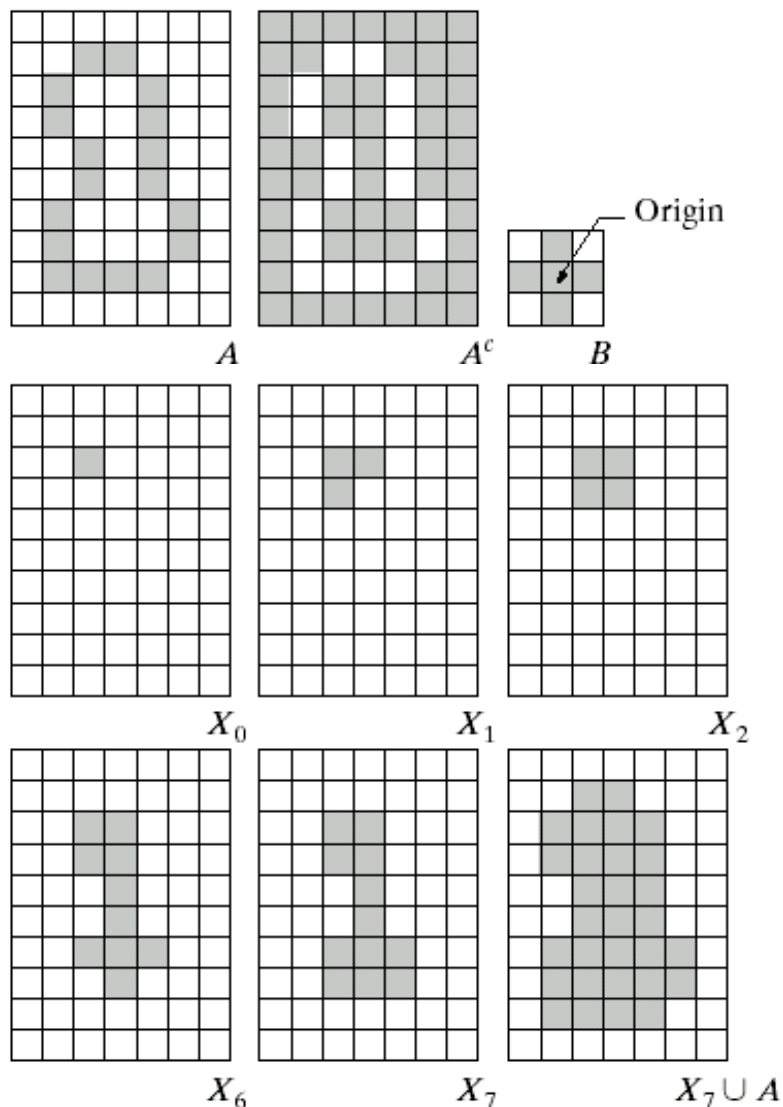


7.5.4 二值形态学图像处理的基本操作

- ◆ **应用实例：** 人形上半身图像侧面轮廓提取



7.5.4 二值形态学图像处理的基本操作



它以集合的膨胀、求补和交集为基础，填充过程如下：

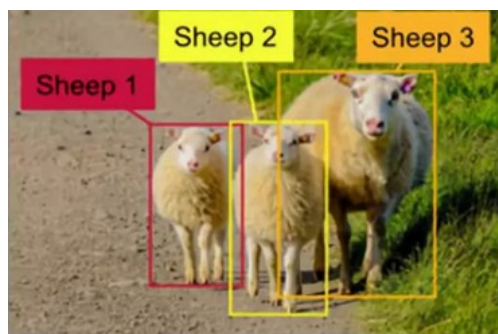
- 将图像中所有的非边界点标记为0；
- 任意选取边界区域内的一点设定为1；
- 以边界点为约束进行条件膨胀处理；
- 通过迭代处理完成对目标区域的填充。

$$X_k = (X_{k-1} \oplus B) \cap A^c \quad k = 1, 2, \dots$$

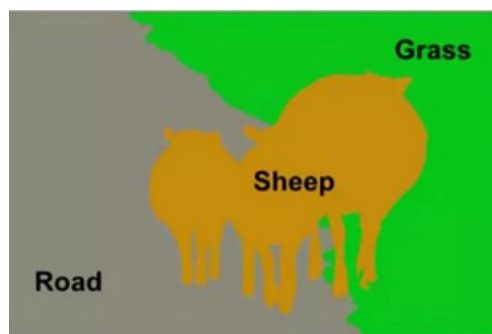
7.6.1 基于深度学习图像分割概述

◆ 基于深度学习的图像分割

- 主要分为两类：语义分割和实例分割。
 - 语义分割：为图像中的每个像素分配一个类别，但是同一类别之间的对象不会区分。
 - 实例分割：对特定的物体进行分类。与目标检测相似，但实例分割输出的是目标的像素级掩码和类别，因此实例分割更加精细，也更有应用价值。



目标检测



语义分割

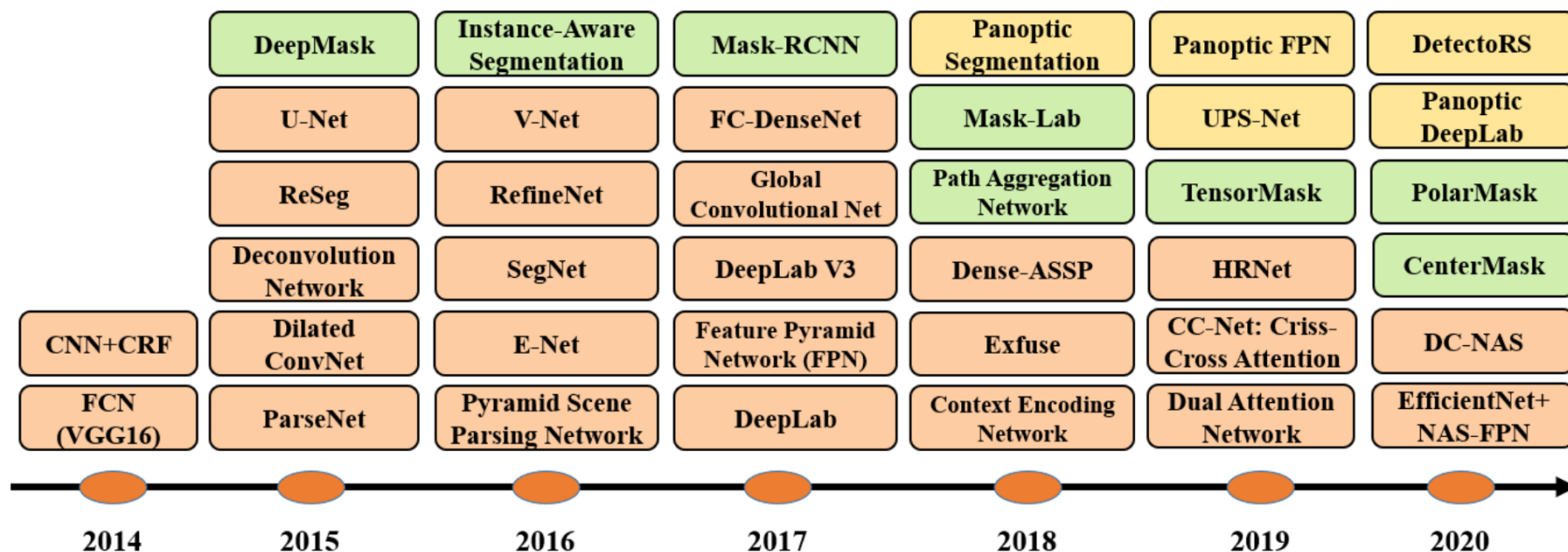


实例分割

7.6.1 基于深度学习图像分割概述

◆ 分割算法发展历史

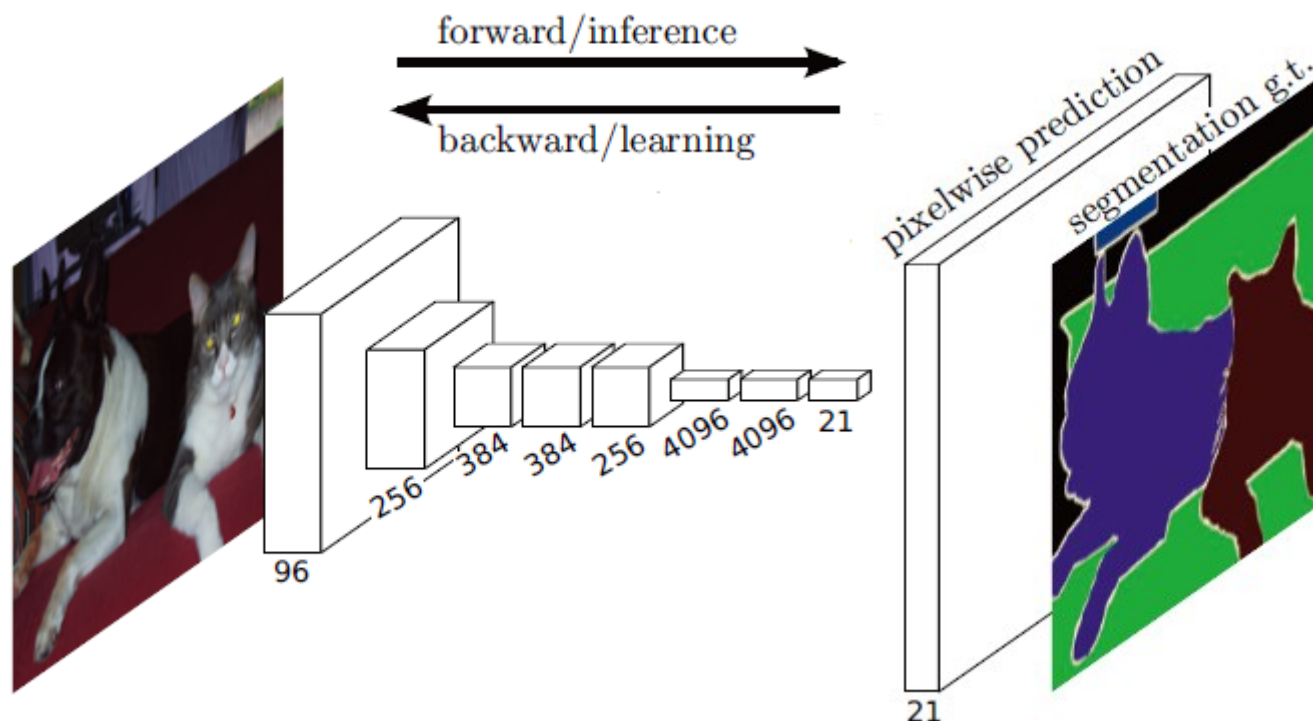
- 图像分割算法在过去的几年里经历了显著的变化，尤其是在卷积神经网络和全卷积网络出现后，图像分割的精度和效率有了质的飞跃。以下是图像分割技术的整体发展历程：



7.6.2 全卷积网络

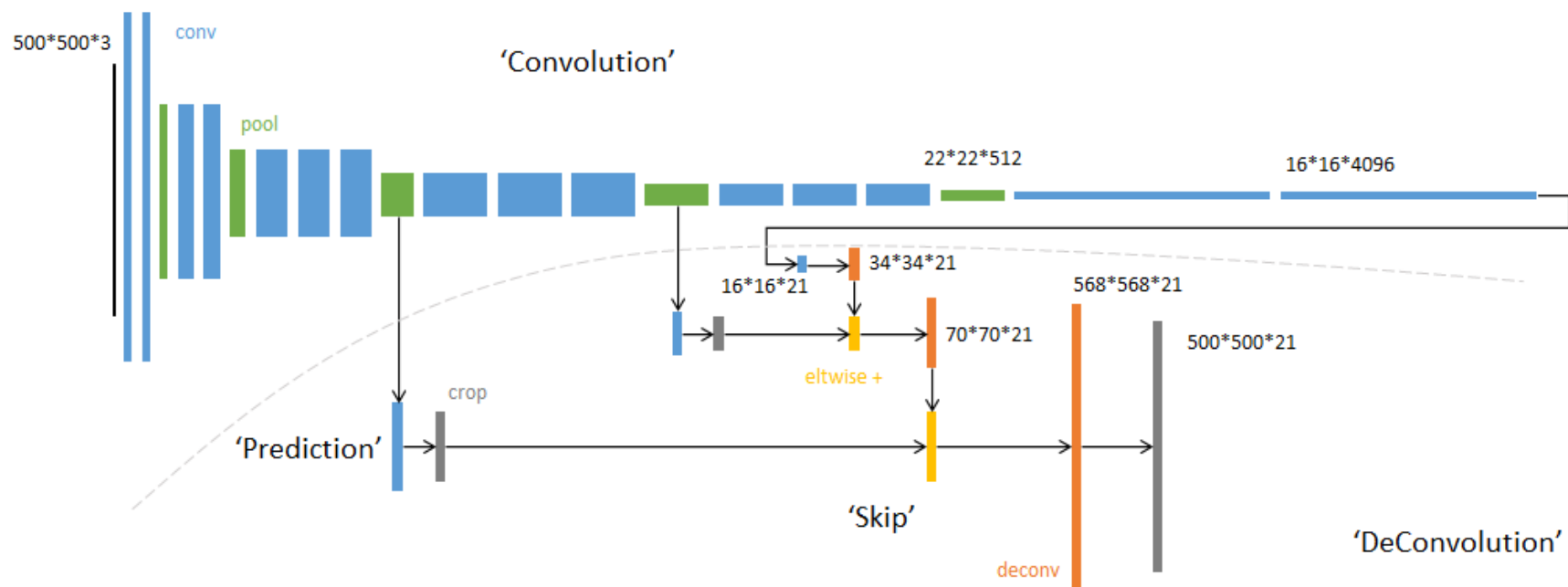
从分类到分割：

全卷积网络（Fully Convolutional Networks, FCN）是深度学习用于语义分割领域的开山之作，首次提出纯全卷积架构，实现了端到端的像素级语义分割预测。



7.6.2 全卷积网络

全卷积网络 (Fully Convolutional Networks, FCN)



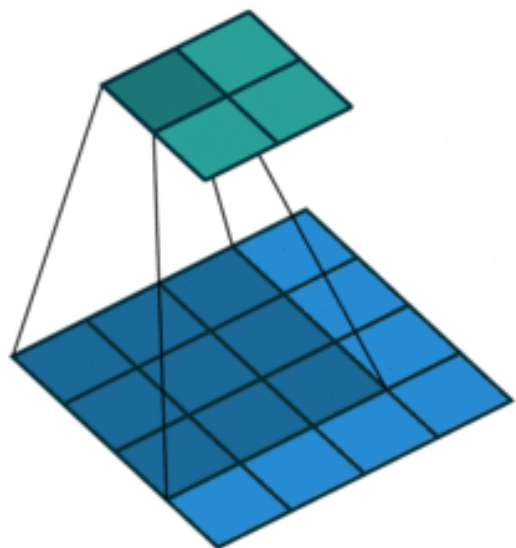
FCN网络结构

FCN网络结构主要分为两个部分：全卷积和反卷积，其中

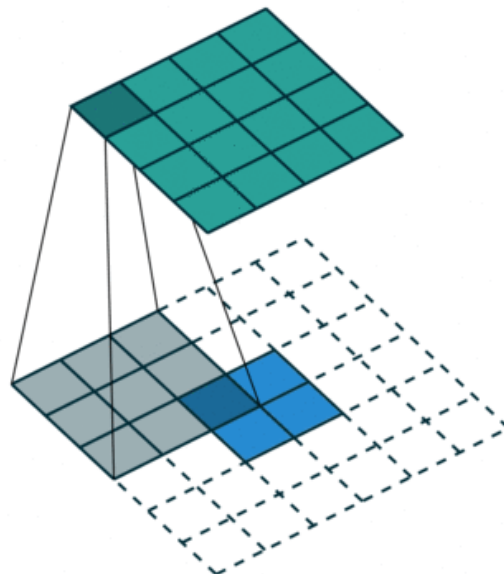
- **全卷积**部分为一些经典的CNN网络（如VGG，ResNet等），用于**提取特征**；
- **反卷积**部分则是通过**上采样**得到原尺寸的语义分割图像。FCN的输入可以为任意尺寸的彩色图像，输出与输入尺寸相同，通道数为 N （目标类别数）+1（背景）。

7.6.2 全卷积网络

上采样方式：反卷积



常规卷积



反卷积

- 反卷积不是常规卷积的逆运算，不能还原常规卷积丢失的信息
- 反卷积可以通过“特征图补零 + 常规卷积”的方式等效实现
- 反卷积输出图像受卷积核尺寸 k 、步距 s 和补零数 p 决定

$$H_{\text{out}} = (H_{\text{in}} - 1) \cdot s + k - 2p$$

7.6.2 全卷积网络

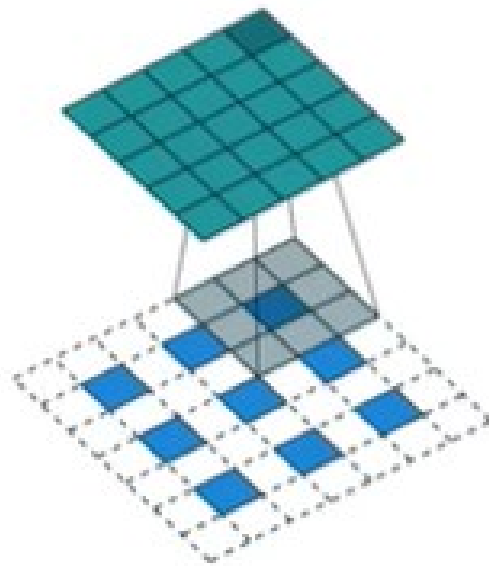
◆ 上采样方式：反卷积

假设输入特征图： $\mathbf{X} \in \mathbf{R}^{H_i \times W_i}$

卷积核： $\mathbf{K} \in \mathbf{R}^{k \times k}$

步长为s，填充为p

转置卷积输出图： $\mathbf{Y} \in \mathbf{R}^{H_o \times W_o}$



$$\mathbf{Y}(x, y) = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} \left(\mathbf{X} \left(\left\lfloor \frac{x-m+p}{s} \right\rfloor, \left\lfloor \frac{y-n+p}{s} \right\rfloor \right) \cdot \mathbf{K}(k-1-m, k-1-n) \right)$$

$$H_o = (H_i - 1) \cdot s + k - 2p$$

$$W_o = (W_i - 1) \cdot s + k - 2p$$

对输入特征图X进行
膨胀，在每个元素
之间插入s-1个零

卷积核需要
上下 / 左右翻转

7.6.2 全卷积网络

上采样方式：反卷积

假设输入特征图： $\mathbf{X} \in \mathbf{R}^{H_i \times W_i}$

卷积核： $\mathbf{K} \in \mathbf{R}^{k \times k}$

步长为 $s=2$ ，填充为 $p=0$

转置卷积输出图： $\mathbf{Y} \in \mathbf{R}^{H_o \times W_o}$

$$\mathbf{X} = \begin{bmatrix} 2 & 3 \\ 1 & 3 \end{bmatrix} \quad \mathbf{K} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 1 & 0 \end{bmatrix}$$

$$H_o = (2-1) \cdot 2 + 3 - 2 \cdot 0 = 5$$

$$W_o = (2-1) \cdot 2 + 3 - 2 \cdot 0 = 5$$

膨胀输入矩阵

$$\mathbf{X}_{up} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 2 & 0 & 3 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 3 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

卷积核反转

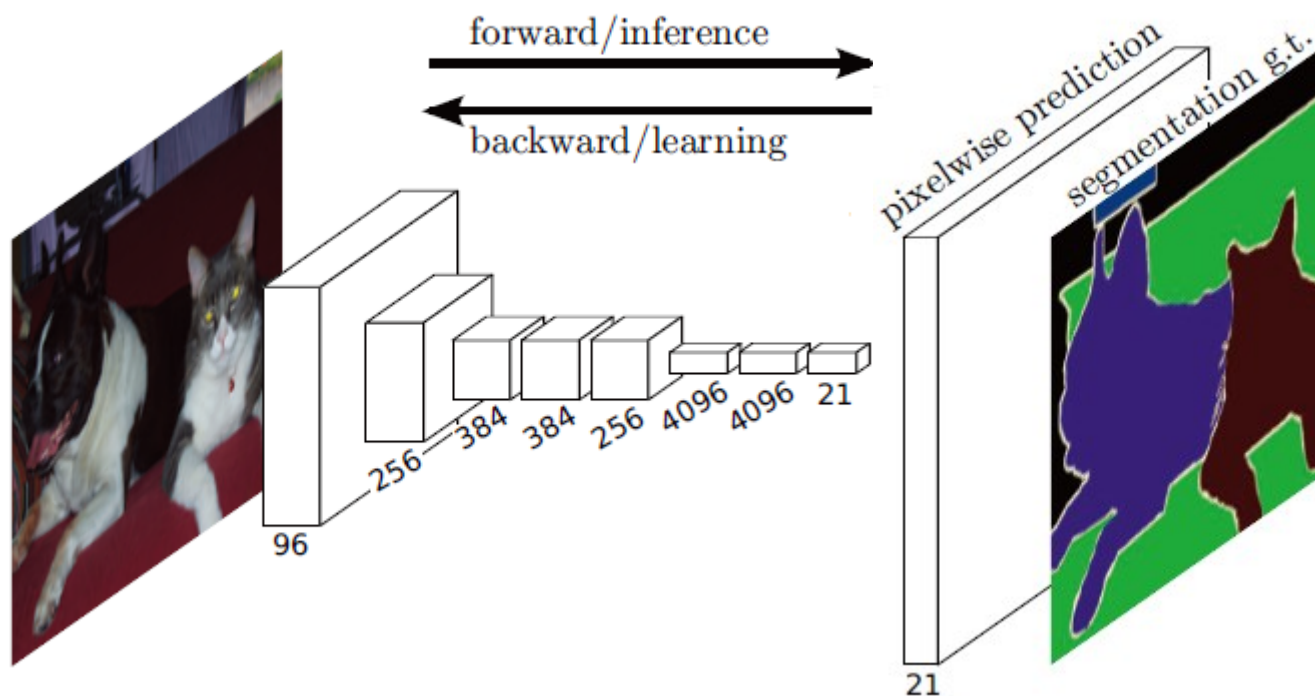
$$\mathbf{K}' = \begin{bmatrix} 0 & 1 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}$$

反卷积处理结果

$$\mathbf{Y} = \begin{bmatrix} 2 & 0 & 5 & 0 & 3 \\ 0 & 2 & 0 & 3 & 0 \\ 3 & 2 & 7 & 3 & 3 \\ 0 & 1 & 0 & 3 & 0 \\ 1 & 1 & 3 & 3 & 0 \end{bmatrix}$$

7.6.2 全卷积网络

◆ FCN 的痛点 —— 粗糙的边缘



FCN的深层特征丢失了空间细节，反卷积上采样易产生边缘模糊，
如何兼顾高层语义特征与底层位置细节？

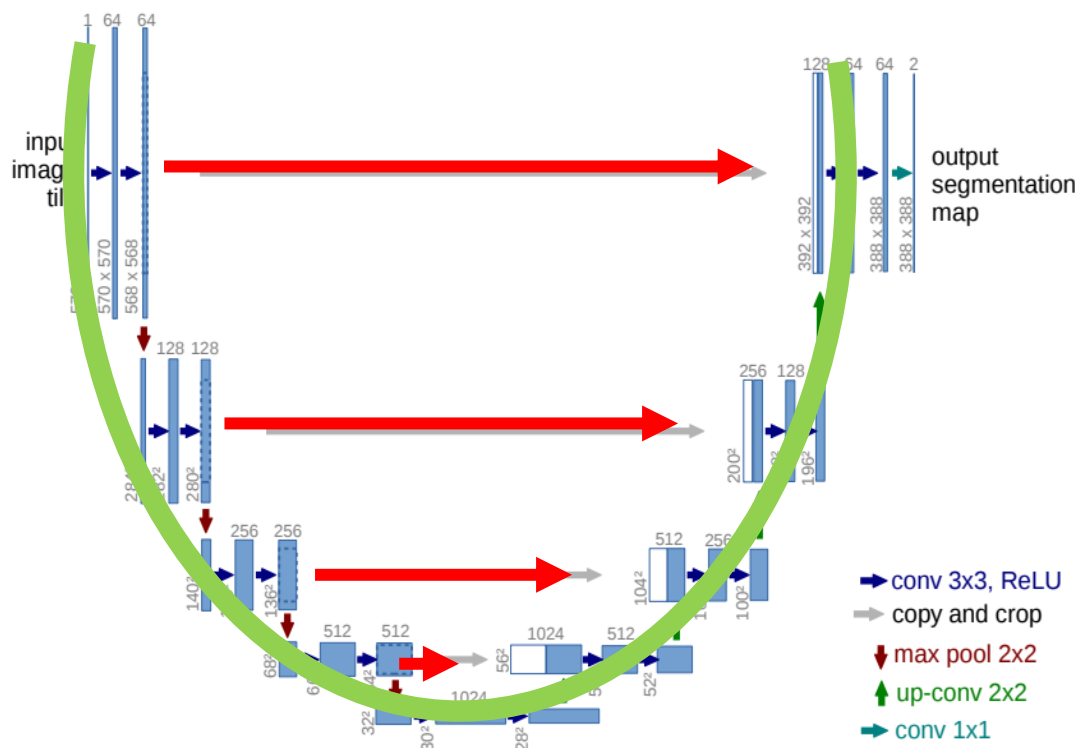
7.6.3 U-Net 网络架构

◆ U-Net 的提出

U-Net由Olaf Ronneberger等在MICCAI 2015上提出，专为医学影像分割设计的全卷积神经网络，是医学图像分割领域的基石模型。

核心架构创新点：

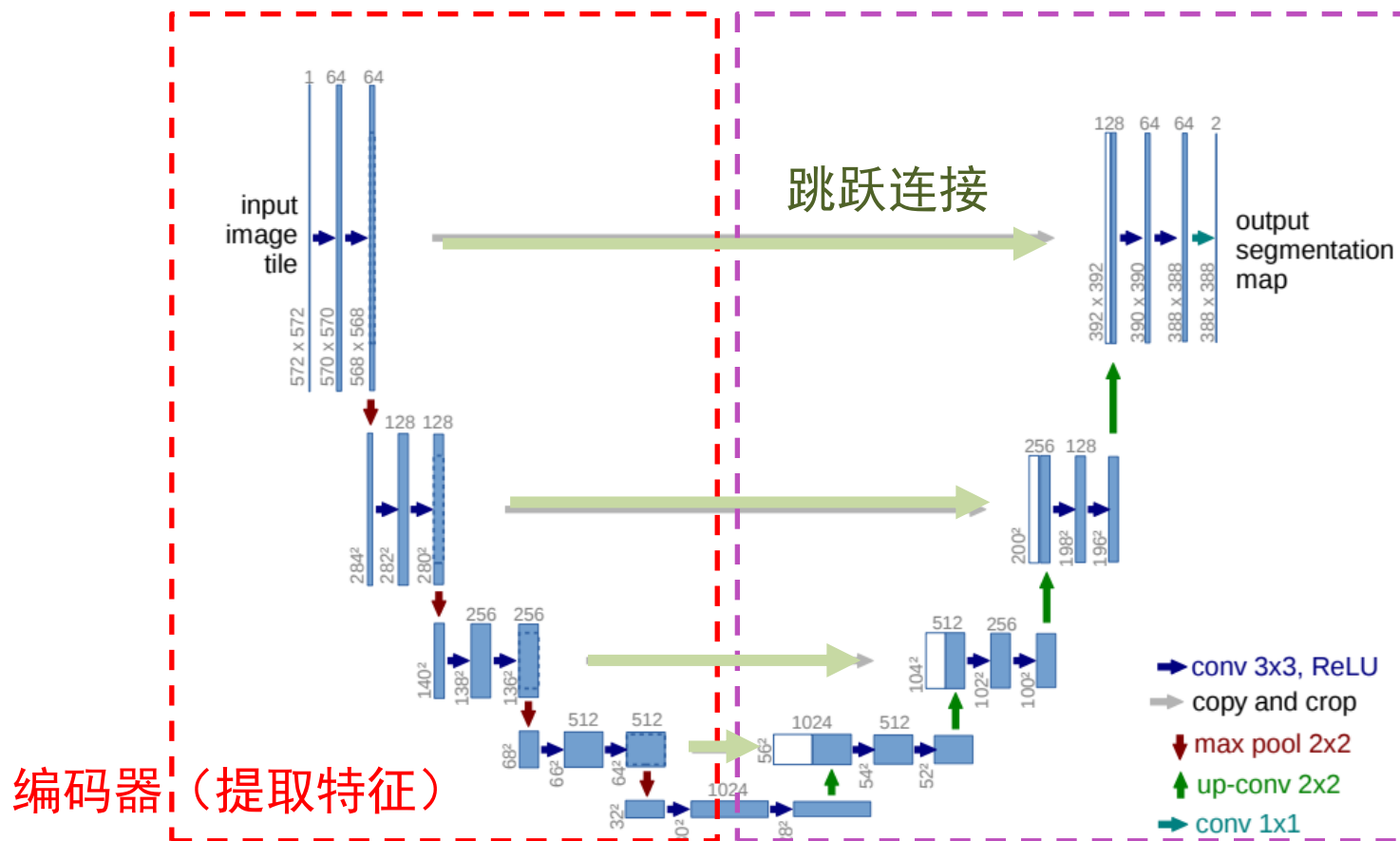
- 对称的 U 型结构
(Encoder-Decoder)
- “跳跃连接”
(Skip Connection)



7.6.3 U-Net 网络架构

◆ 模型总体概览

解码器（上采样、恢复空间分辨率）



7.6.3 U-Net 网络架构

- 编码器/Encoder: 下采样与高级特征提取

- 宽度和高度 (H * W)越来越小:

572→(conv)568→(pool)284→(conv)280

→(pool)140→(conv)136→(pool)68

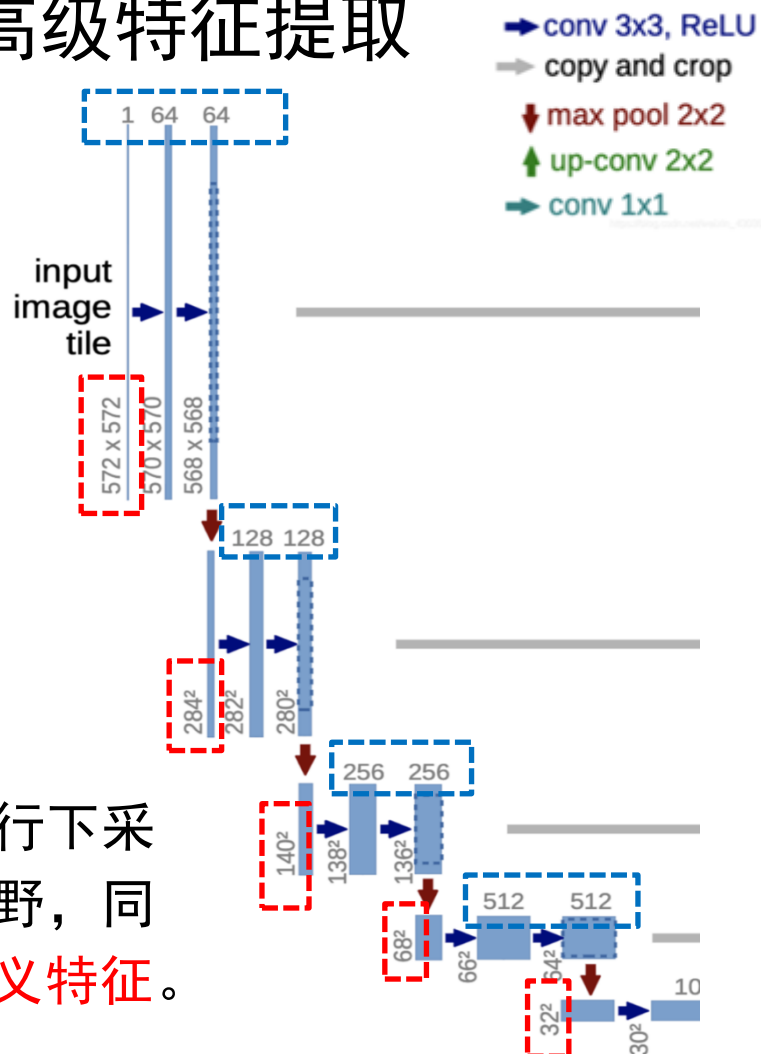
→(conv)64→(pool)32→(conv)28

- 通道数 (C) 越来越深

64→128→256→512→1024

编码器操作本质上是

反复执行“双卷积块”，并通过最大池化进行下采样，逐步降低特征图空间尺寸、扩大感受野，同时将通道数翻倍，**提取从低层到高层的语义特征。**



7.6.3 U-Net 网络架构

◆ 解码器/Decoder: 上采样与空间分辨率恢复

- 宽度和高度 ($H * W$) 越来越大:

$28 \rightarrow (\text{up-conv}) 56 \rightarrow (\text{double } 3 \times 3 \text{ conv}) 52$

$\rightarrow (\text{up-conv}) 104 \rightarrow (\text{double } 3 \times 3 \text{ conv}) 100$

$\rightarrow (\text{up-conv}) 200 \rightarrow (\text{double } 3 \times 3 \text{ conv}) 196$

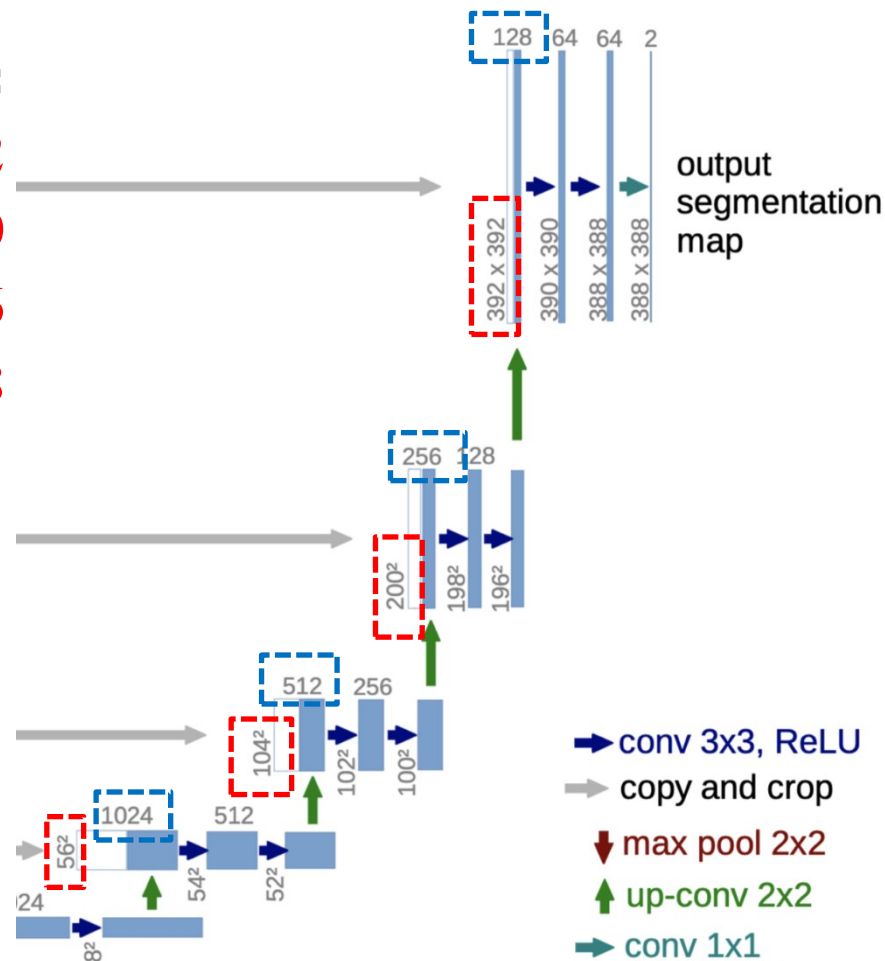
$\rightarrow (\text{up-conv}) 392 \rightarrow (\text{double } 3 \times 3 \text{ conv}) 388$

- 通道数 (C) 越来越浅

$1024 \rightarrow 512 \rightarrow 256 \rightarrow 128$

编码器操作本质上是:

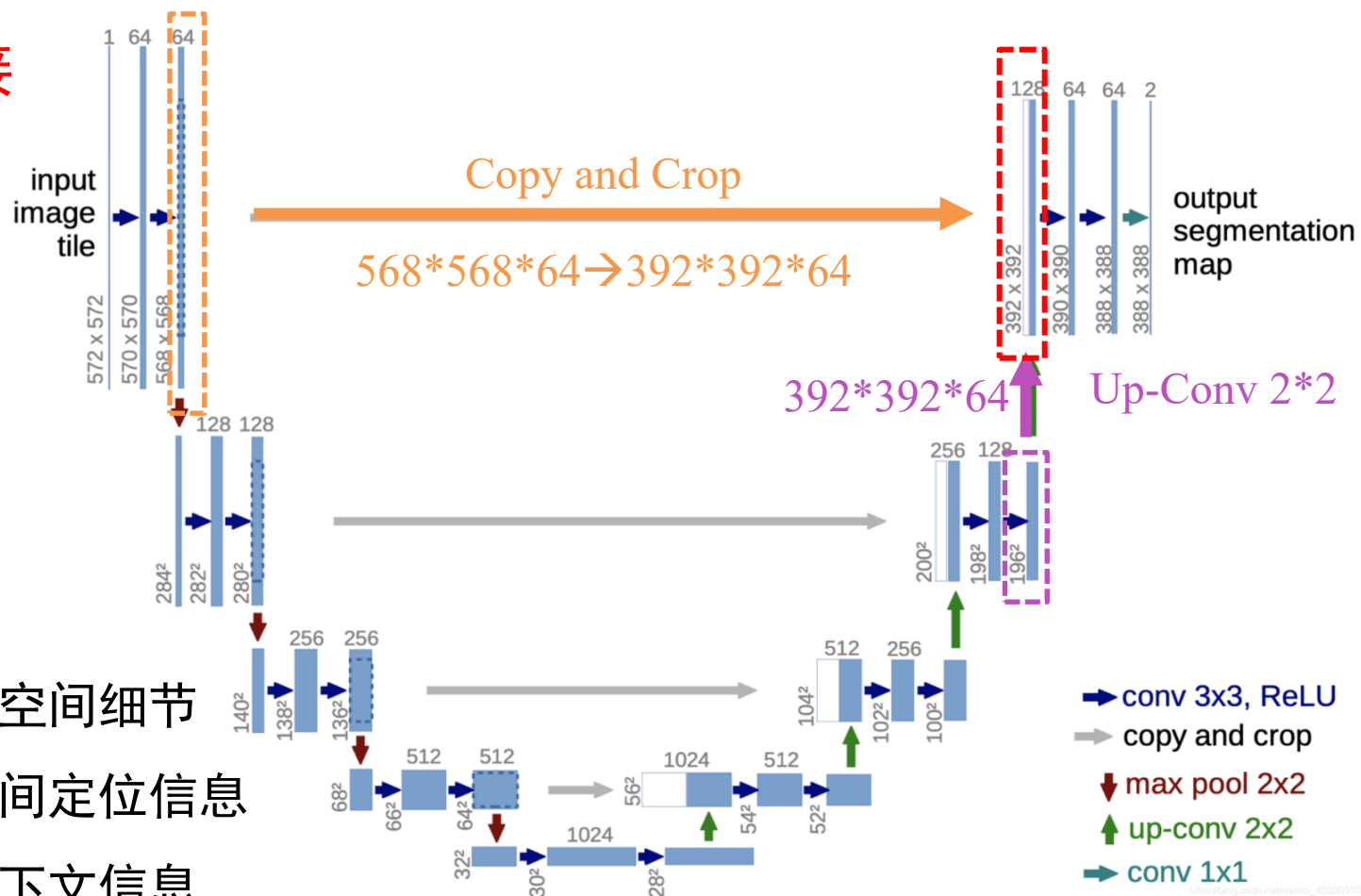
交替执行反卷积与“双卷积块”，逐步恢复特征图的空间分辨率，同时降低通道数，将高维语义特征映射回像素级空间！



7.6.3 U-Net 网络架构

跳跃连接 (Skip Connections)

Concatenate 拼接 (通道数相加)



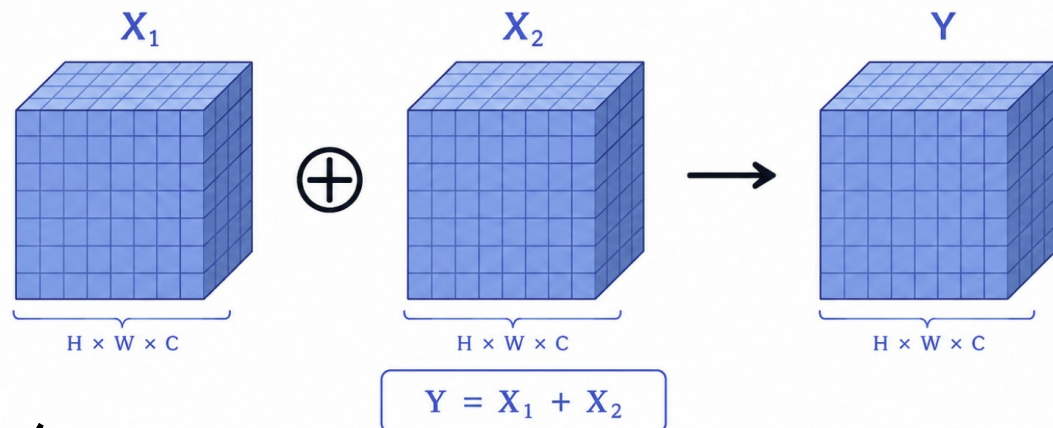
跳跃连接引入了

- 1) 引入高分辨率空间细节
- 2) 提供精准的空间定位信息
- 3) 融合多尺度上下文信息

7.6.3 U-Net 网络架构

语义分割网络在特征融合时的两种办法

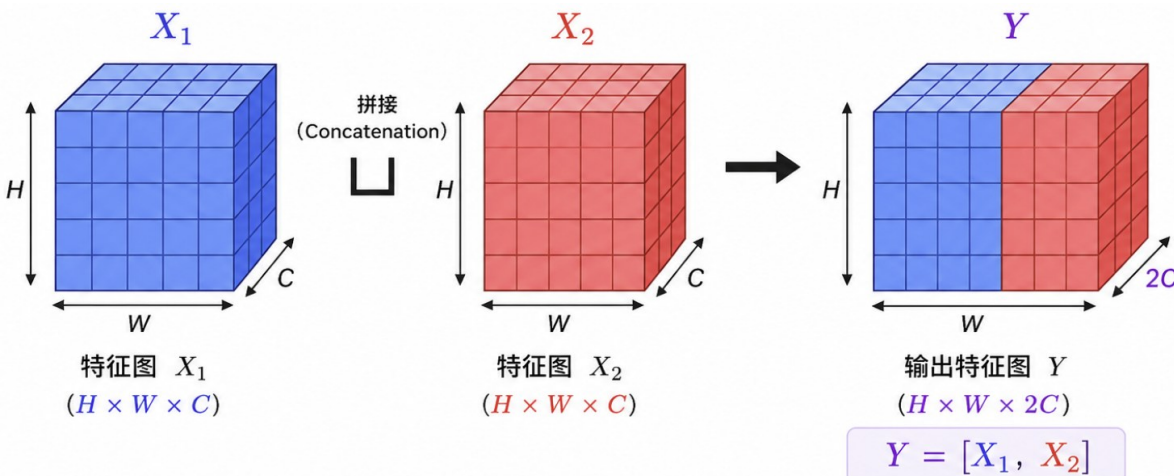
1. FCN式的对应点相加-Add



2. U-Net式的特征维度拼接融合-Concatenate

拼接(Concat)通过通道堆叠实现

无损融合，在确保原始细节留存的同时，赋予网络**自主筛选**细节与语义的权利，从而显著提升了复杂目标的定位精度



7.6.3 U-Net 网络架构

◆ 损失函数：标准交叉熵损失函数

标准的**多类别交叉熵损失函数**(Multi-class Cross-Entropy)

$$Loss = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log(p_{i,c})$$

- N ：总像素数
- C ：总类别数（如：背景、车道线、行人等）
- $y_{i,c}$ ：真实标签
- $p_{i,c}$ ：网络预测第*i*个像素为第*c*个类别的概率

交叉熵损失的目标是**最小化Loss值**，使得模型输出的预测概率与真实标签的分布尽可能接近。在这里是指将整张图片的分割任务，拆解为 N 个独立的像素级多分类选择题，所有像素的误差求和取平均。

在类别不平衡的分割任务中，标准交叉熵损失存在明显缺陷：

对所有像素损失同等对待，模型会优先优化占比高的主体区域，忽略占比极低的边界和小目标像素，导致分割边界模糊、目标合并！

7.6.3 U-Net 网络架构

◆ 损失函数：加权交叉熵（Weighted Cross-Entropy）

U-Net采用**加权交叉熵损失函数**，首先来看计算过程

$$E = \sum_{\mathbf{x} \in \Omega} w(\mathbf{x}) \log(p_{l(\mathbf{x})}(\mathbf{x}))$$

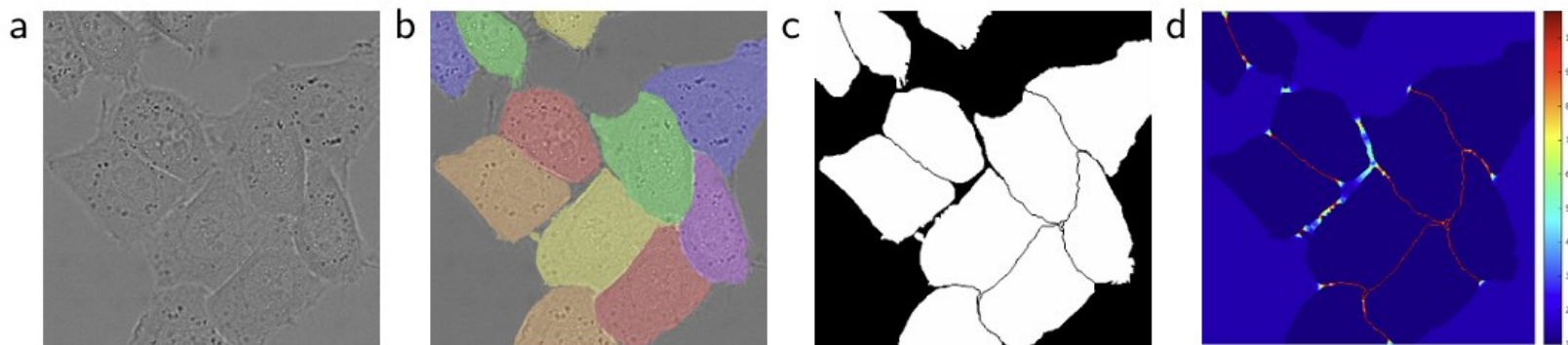
- $p_{l(x)}$ ：模型预测的样本属于真实标签的概率 $l(x)$ ：真实标签
- $\log(p_{l(x)})$ ：网络预测正确的概率 $w(x)$ ：赋予每个像素 x 的惩罚倍率(权重)

$$w(\mathbf{x}) = w_c(\mathbf{x}) + w_0 \cdot \exp\left(-\frac{(d_1(\mathbf{x}) + d_2(\mathbf{x}))^2}{2\sigma^2}\right)$$

- $w_c(x)$ ：**解决“类别不平衡”的权重参数**，通常直接统计各个类别的像素占比后取倒数
- w_0 ：最高惩罚倍数（原文设为10） σ ：惩罚范围的宽度(原文约为5个像素)
- $d_1(x)$ ：当前像素点走到离它最近的一个目标边界的距离，和 $d_2(x)$ 都是提前计算的
- $d_2(x)$ ：当前像素点走到离它第二近的另一个目标边界的距离

7.6.3 U-Net 网络架构

◆ 损失函数：加权交叉熵（Weighted Cross-Entropy）



(a) 原始细胞图 (b) 细胞的真实标签图 (c) 二值分割结果 (d) 热力权重图

- **核心问题：**同类目标密集排布时极易边界漏检；目标与背景像素比例严重失衡
- **U-Net 使用加权交叉熵：**为每个像素定制权重 $w(x)$ ，强制网络聚焦于“相互接触的细胞之间的狭窄背景”，做错给予极高惩罚

$$E = \sum_{\mathbf{x} \in \Omega} w(\mathbf{x}) \log(p_{\ell(\mathbf{x})}(\mathbf{x}))$$

$$w(\mathbf{x}) = w_c(\mathbf{x}) + w_0 \cdot \exp\left(-\frac{(d_1(\mathbf{x}) + d_2(\mathbf{x}))^2}{2\sigma^2}\right)$$

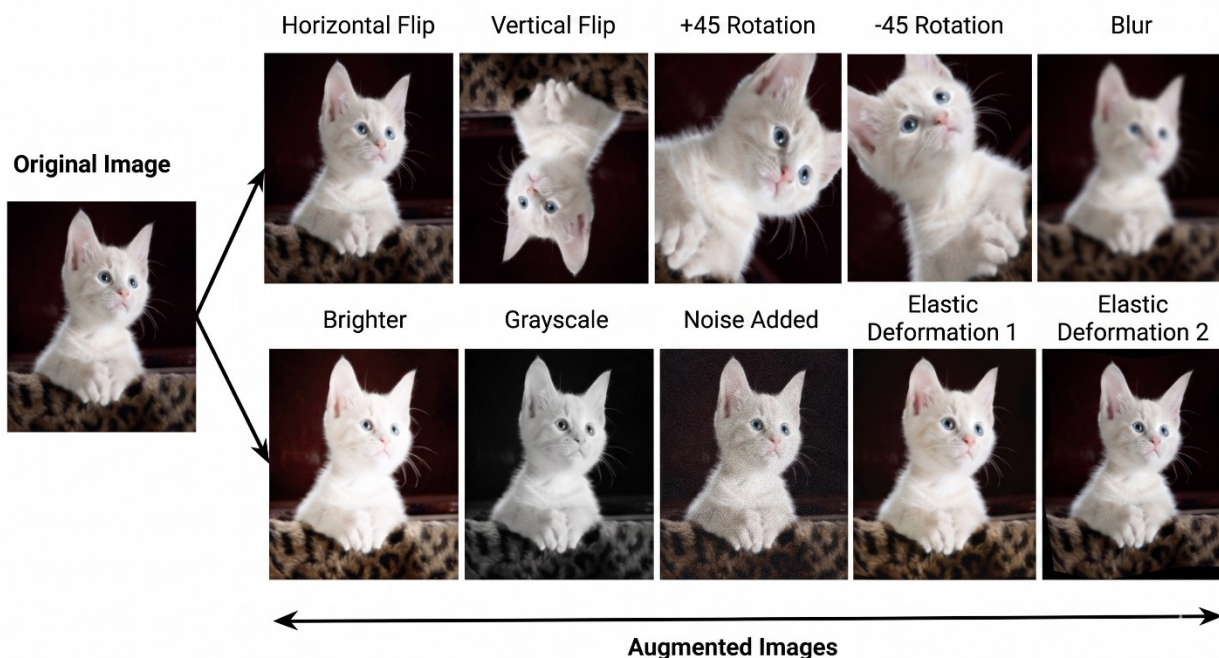
- 为每一个真实分割预先计算了权重图 $w(x)$ ，以补偿训练集里某个类别的像素的不同频率，并且迫使网络学习相邻细胞间的引入小的分割边界（见图c和d）。
- 计算像素到最近和第二近目标的距离 $(d_1 + d_2)$ 。距离越近的背景夹缝，赋予越极端的惩罚权重（图中的深红色高亮线条）。

7.6.3 U-Net 网络架构

◆ 训练相关：数据增强（Data Augmentation）

数据增强：是通过对现有数据**施加合理变换来生成新样本**，以扩充数据集规模、增加多样性，最终提升模型泛化能力、防止过拟合的核心技术。

常规方法：包括旋转、反转、弹性形变和灰度值变化等

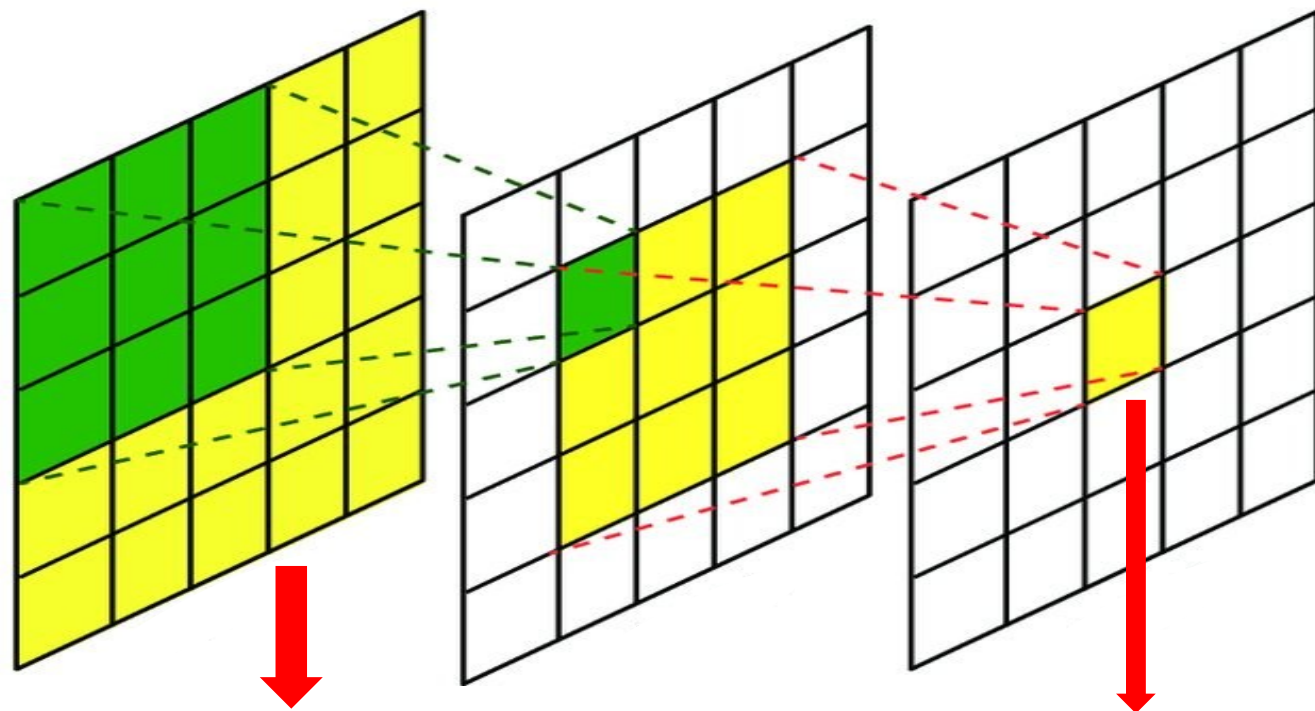


U-Net 数据增强：弹性形变（Elastic Deformations）

原理：利用粗糙网格和高斯滤波器产生随机位移向量，模拟生物组织的真实形变。

7.6.3 U-Net 网络架构

感受野与定位精度的权衡



Layer1 (浅层：感受野小)

Layer3 (深层：感受野大)

感受野(Receptive Field):

在卷积神经网络中，某一层特征图 (Feature Map) 上的一个像素点，它所对应的原始输入图像上的区域大小

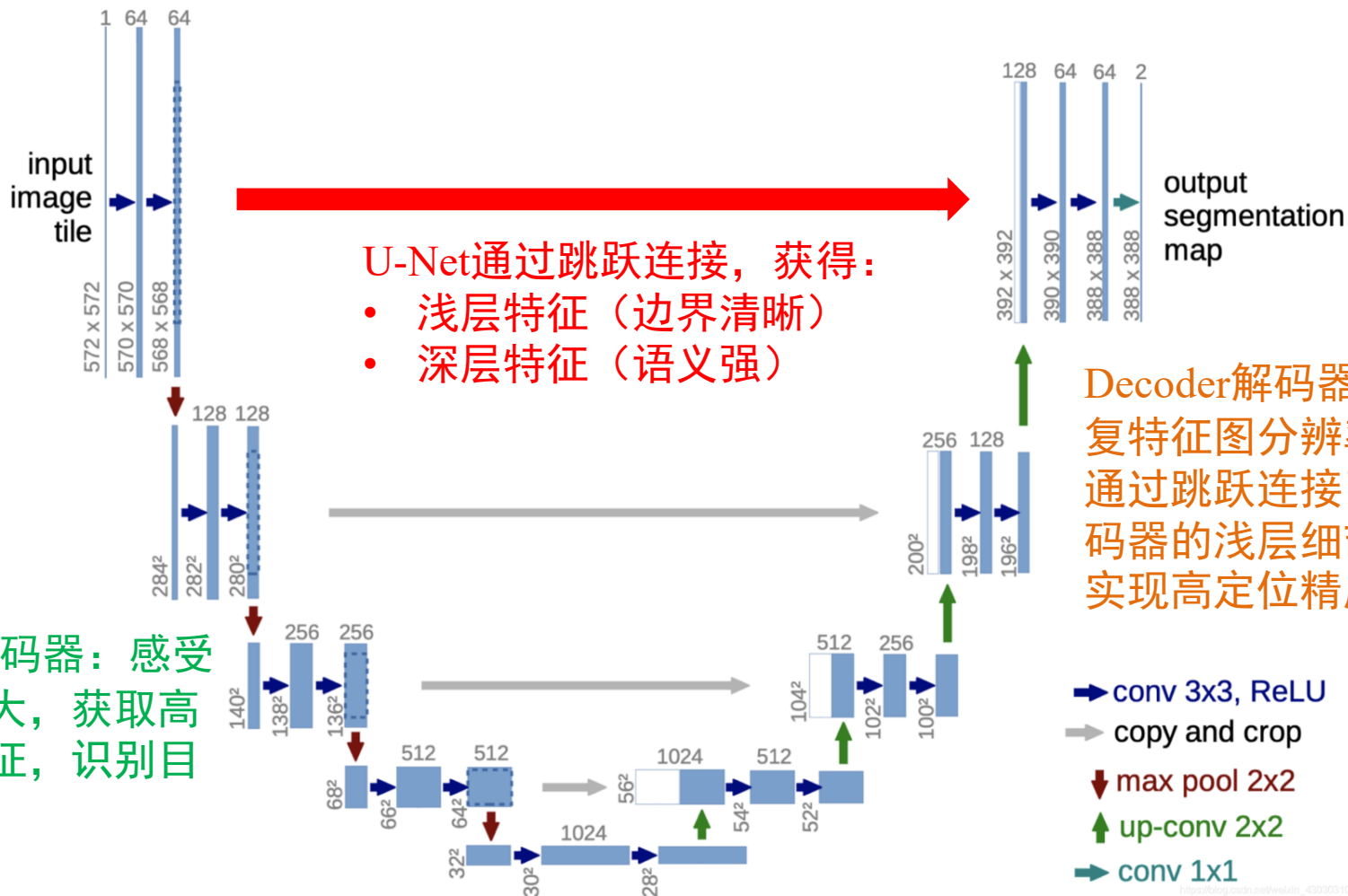
定位精度(Localization Accuracy):

模型在处理视觉任务时，能够精确划分出目标物体空间边界 (或坐标位置) 的能力

- 深层网络：感受野大 → 语义强 (**能认出目标**) → 但定位模糊
- 浅层网络：感受野小 → 边缘清晰 (**定位准**) → 但缺乏全局认知

7.6.3 U-Net 网络架构

感受野与定位精度的权衡



7.6.3 U-Net 网络架构

为什么U-Net适合小数据集？

✓ 全卷积架构与参数效率：

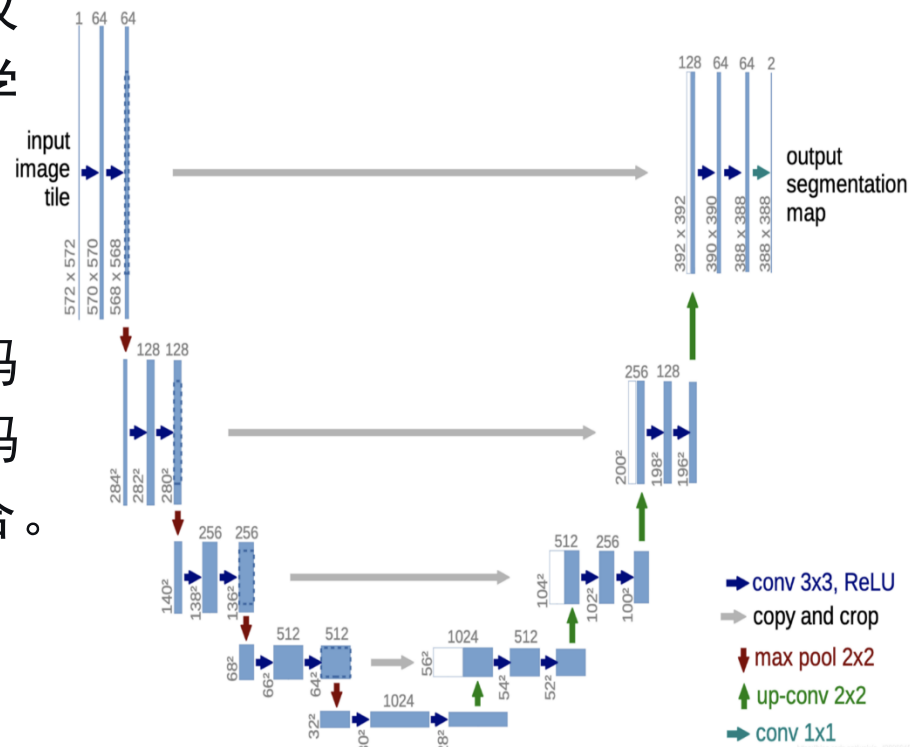
U-Net采用全卷积架构，继承了CNN的权重共享特性，在有限样本下仍能有效学习，降低了对大规模标注数据的依赖。

✓ U形结构与跳跃连接：

独特的U形结构与跳跃连接设计，将编码器的高分辨率空间细节直接传递至解码器，实现了低层细节与高层语义的融合。

✓ 数据增强策略：

通过对有限样本进行扩充，增强模型对形态变化的泛化能力。

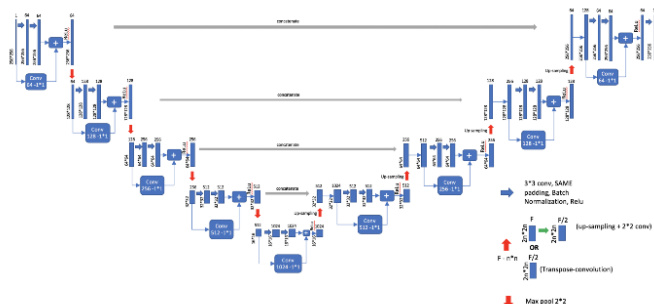


7.6.3 U-Net 网络架构

◆ 主要改进架构

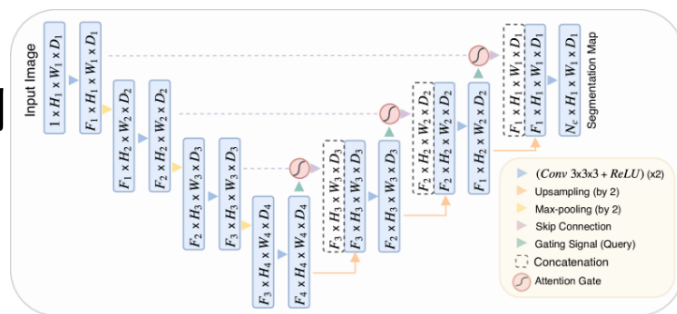
➤ ResUNet:

深层网络容易出现**梯度消失问题**，ResUNet在U-Net的基础上，将编码器和解码器中引入**残差连接**，有效缓解了梯度消失问题，同时增强了特征复用能力，让深层网络更易训练和优化。



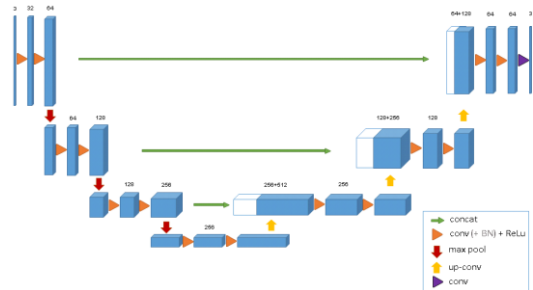
➤ Attention U-Net:

在常规 U-Net 中，跳跃连接将**所有特征**从Encoder传递到Decoder。因此2018年，Ozan等人在跳跃连接处引入 Attention Gate，**可以自动聚焦目标区域，抑制无关背景**



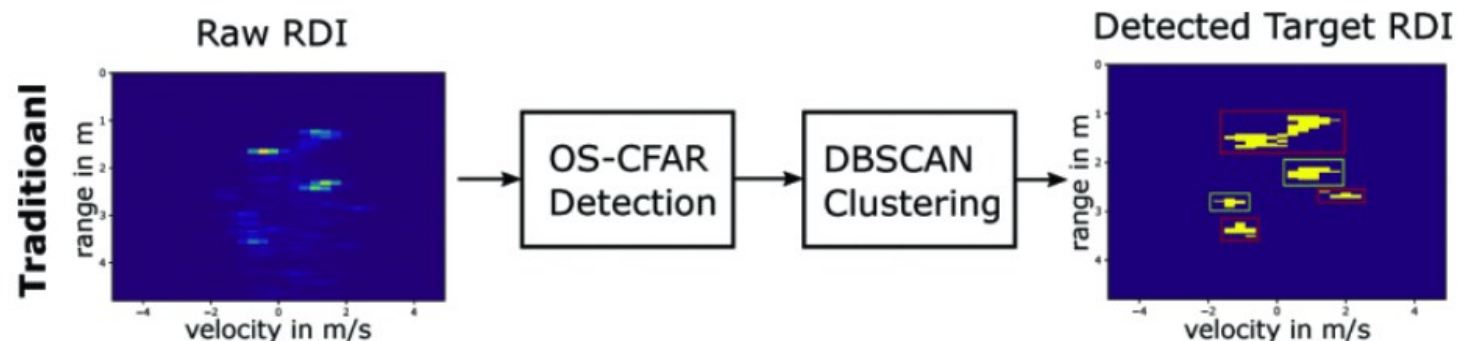
➤ 3D U-Net:

许多数据本质是**三维体数据**，由一系列连续切片构成立体结构。传统 2D U-Net 逐切片处理，会丢失 Z 轴上下文信息，难以捕捉目标的三维连续结构。因此，Olaf (U-Net原作者) 于2016年引入3D U-Net，实现端到端的**三维体数据分割**。

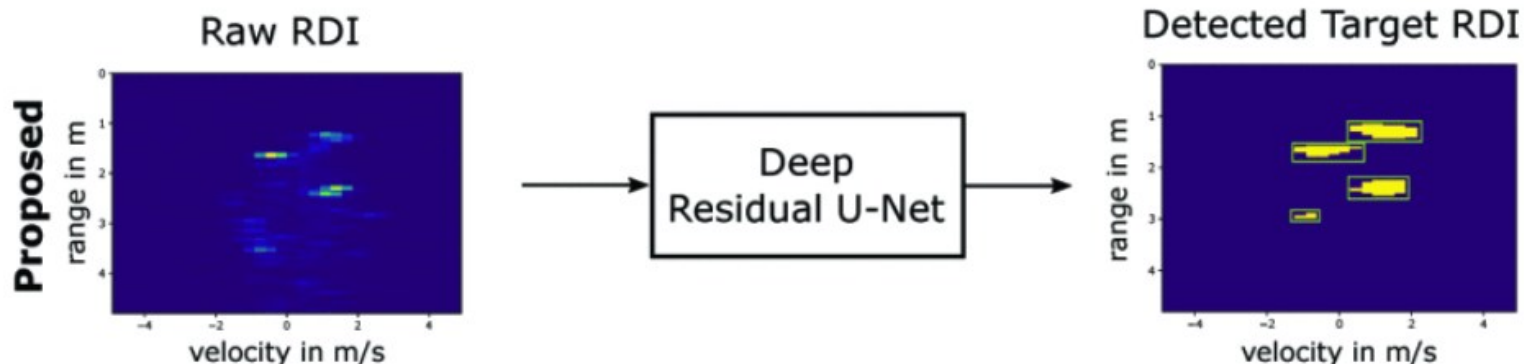


7.6.4 U-Net 的应用与实践

◆ 雷达信号目标检测



Traditional Processing Pipeline (MTI滤波→积累→CFAR→聚类)

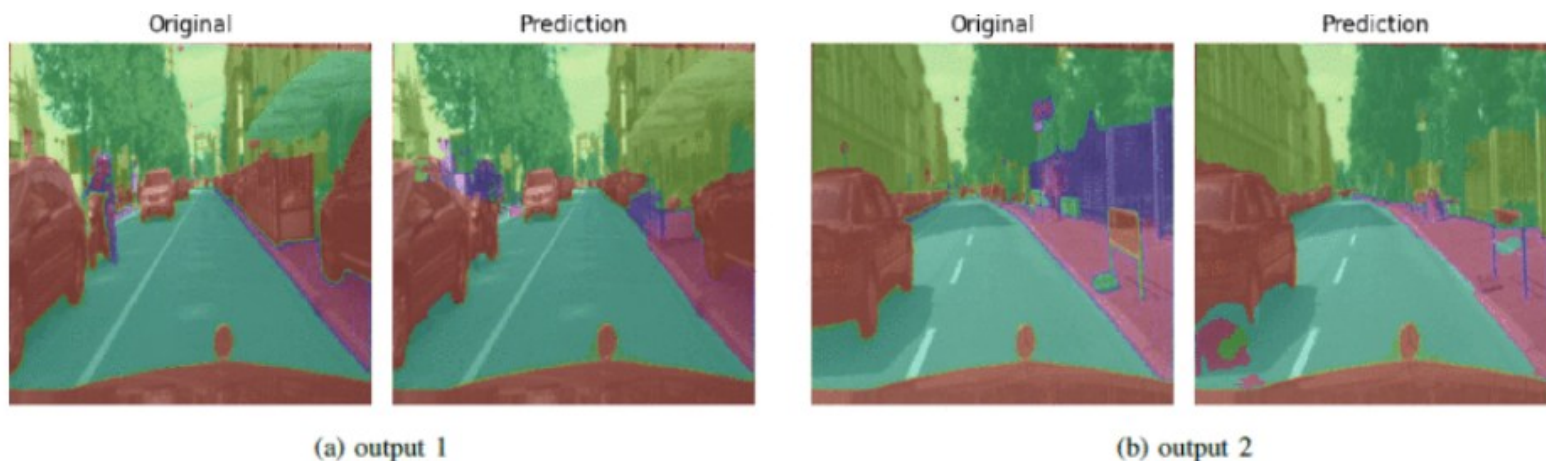
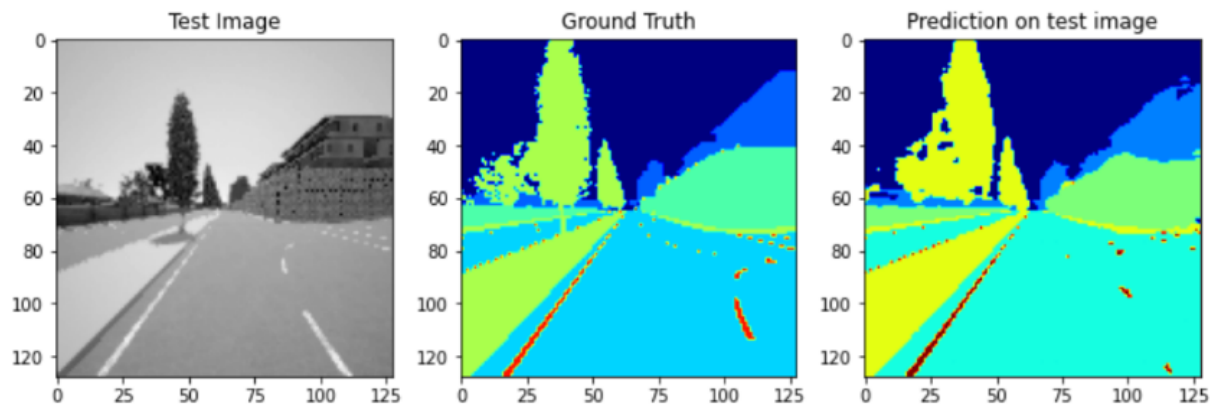


Deep Residual U-Net (端到端检测)

M. Stephan and A. Santra, "Radar-Based Human Target Detection using Deep Residual U-Net for Smart Home Applications," 2019 18th IEEE International Conference On Machine Learning And Applications (ICMLA), Boca Raton, FL, USA, 2019, pp. 175-182.

7.6.4 U-Net 网络的应用与实践

◆ 自动驾驶



V. S. Natta, J. Kadicherla, S. D. Katukojwala, K. Swaraja, N. A. Vignesh and S. Aouthu, "Semantic Segmentation Using U-Net for Autonomous Driving," 2024 Third International Conference on Distributed Computing and Electrical Circuits and Electronics, Ballari, India, 2024, pp. 1-6,

作业

7.1 图像矩阵 A (6×6 , 二值图像, 0 表示背景, 1 表示前景), 结构元素 B (3×3 十字形, 原点在中心), 计算 A 被 B 腐蚀的结果矩阵 (尺寸与 A 相同, 边界外的像素视为背景 0)。

0	0	0	0	0	0
0	1	1	1	1	0
0	1	1	1	1	0
0	1	1	1	1	0
0	1	1	1	1	0
0	0	0	0	0	0

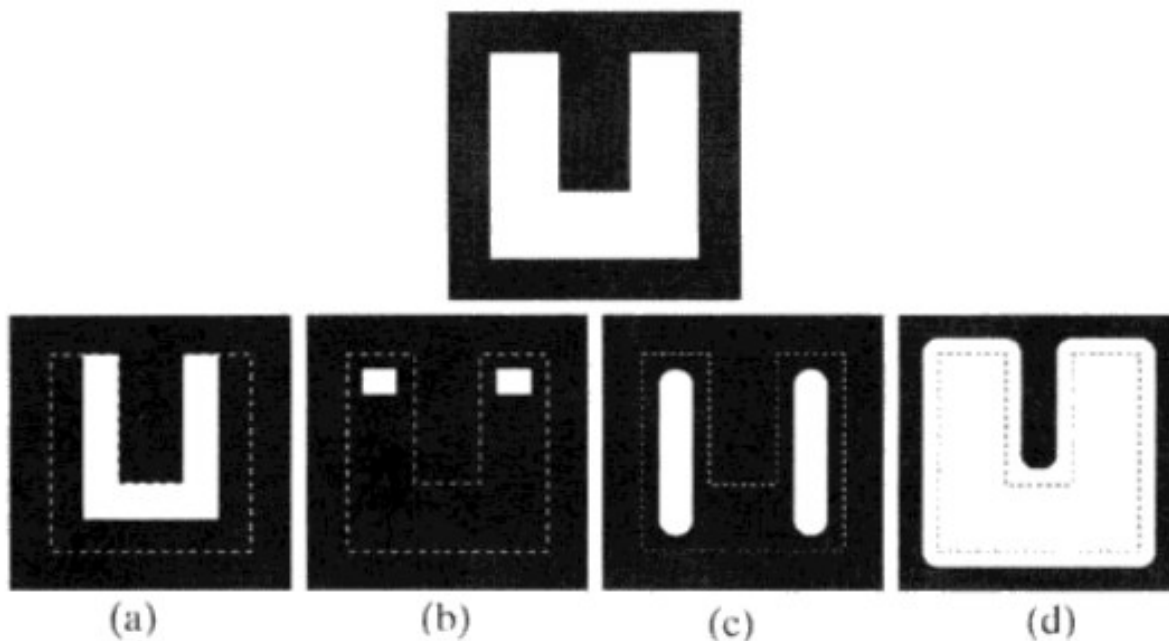
图像矩阵 A

0	1	0
1	1	1
0	1	0

结构元素 B

作业

7.2 参考所显示的图像，给出生成图(a)至图(d)所示结果的结构元和形态学操作。请清晰的说明每个结构元的圆点。图中的虚线表示原始集合的边界，仅供参考显示。注意图(d)中的所有角都是圆角。



作业

7.3 为什么 U-Net 要在解码器中使用跳跃连接，将编码器的特征直接拼接过来？如果去掉跳跃连接，分割效果可能会出现什么问题？

7.4 为什么 U-Net 采用编码器不断下采样、解码器不断上采样的 U 型对称结构，而不是直接用单层卷积做分割？

7.5 对如下输入 X 计算反卷积：(stride=2, padding=0)

$$X = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

$$\text{Kernel} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

谢 谢！